HOUR: 720
A MASSIVELY MULTIPLAYER EXPERIENCE

by

Shawn Rider
September 1, 2006

A thesis submitted to the
Faculty of the Graduate School of
the State University of New York at Buffalo
in partial fulfillment of the requirements for the
degree of

Master of Fine Arts

Department of Media Study

Acknowledgments

I would like to thank the members of my committee, Josephine Anstey, Loss Pequeño Glazier, and Marc Böhlen, for their efforts supporting this thesis. I appreciate all you've done for me in so many ways. This project could not be completed without the growing ranks of beta testers who have spent time helping to find bugs. I appreciate your enthusiasm and willingness to keep testing. I must also thank Paul Visco, creator of the Surebert AJAX library, for his continued support and many educational lessons that have allowed me to create this project. Sincere thanks and appreciation go out to Steffan Del Piano, who has been a strong collaborator and without whom this project would not have been undertaken. Finally, I must offer my deepest thanks and undying love to Sarah Wichlacz, my wife, collaborator, and best friend, who has been patient, tolerant, and supportive to a degree I'm unsure I'll ever be able to repay.

Table of Contents

Acknowledgments ii

Abstract v

Chapter One:
The Monster-Slayer in the Mirror 7

A brief history of online worlds 9
The conventions of the MMORPG 13
The experimental game goals of Hour: 720 19

Chapter Two:
Welcome to the Disaster Zone 26

Getting into the game 27
An island in crisis 30
A summary of player experience in Hour: 720 32

Chapter Three:
Raising the Dead 47

Technical decisions underlying Hour: 720 50
Structural overview of Hour: 720 52
Beta testing the dead 58

Chapter Four:
Documenting the Play 64

The intersection of games and literature 65
Player created and player edited texts 67
The spectator experience 71
Serialized game texts 73

Appendix..... 76

Database Outline Document ver 0.3..... 76
Code Samples.....81
Sample output from eventParser.php..... 84

Works Cited and Consulted..... 86

Abstract

The production component of the MFA thesis will be a massively-multiplayer role-playing game (MMORPG) called *Hour: 720*. Designed for up to 1000 players to engage with simultaneously, *Hour: 720* is a complex software project that can be installed on any system meeting the basic system requirements. Users do not have to run the software to engage with it; rather, players will engage with the game via any contemporary web browser. The software, including complete source code, will be delivered to the school on disc.

Hour: 720 takes place on an island that has suffered a strange kind of natural disaster: The dead have risen. Zombies are crawling out of the ground, and players find themselves playing survivors who desperately must try to either escape, hide out in hopes of rescue, or otherwise make their way in a world full of the walking dead. The zombie motif is a much-exploited element of game narratives, but developers almost never made use of the rich philosophical potential embedded in the notion of the zombie—the othered self. The zombie motif allows *Hour: 720* to make meaningful abstraction of significant human struggles with identity, community, consumption and social interaction.

Hour: 720 is designed to raise these social issues. For example, in order to repair an airplane and escape from the island, players must band together. Players might need to work extra hard to protect another player who is a retired pilot and needs daily insulin

shots to stay alive. Another option for some players might be to forsake the old man and head for the beach to find or build a boat. By creating a dynamic world with finite resources, as well as characters with true strengths and weaknesses, *Hour: 720* prompts a variety of compelling social action.

Hour: 720 seeks to broaden the notion of what an online gaming experience can be, turning away from most MMORPG conventions. *Hour: 720* is also more than a game: It is a social narrative engine. Every narrative generated is created by live interaction with the simulated world. In order to push interesting social reactions, the game is designed to be a realistic simulation. In addition, outside web users may peek into the world of *Hour: 720* via the public website.

The written component of the MFA thesis will contextualize and explain the design decisions made in the creation of *Hour: 720*. The thesis will discuss some of the interesting social interactions prompted by *Hour: 720*, and inform the reader of the conventions in game design which *Hour: 720* is directly confronting and either rejecting or reworking. Finally, the thesis will address the role of the player in generating narrative components and the relationship between game-generated elements and player-created or player-edited elements.

Chapter One:
The Monster-Slayer in the Mirror

An introduction to MMORPGs.

Hour: 720 is a massively multiplayer role-playing game (MMORPG), if categorized in mainstream gaming terms. It is created and programmed by Shawn Rider with story and rules assistance from Steffan Del Piano. While it would be inaccurate to call *Hour: 720* a "traditional" MMORPG, it is also necessary to understand the landscape of mainstream MMORPGs in order to understand the ways in which *Hour: 720* tweaks the genre to create a fiction that blends several genres and forms in hopes of creating a compelling interactive experience.

An observation of contemporary game worlds reveals that players are deeply involved with these games. Avatars are imbued with deep meaning. School-age kids spend hours engaging in research, documentation and writing in order to better interact with the world. Huge numbers of people collaborate to create complex systems of government and bureaucracy. Imaginary companies are formed that sometimes earn real-world profits. The question of how compelling an online game world can be is not

answered by numbers of users or sales figures, but by the actions of thousands of players who are deeply invested in their game world experience.

It would be impossible to go into a full study of user-behaviors in online games in this thesis, but one need only look at the work of scholars such as Sherry Turkle and Constance Steinkuehler to gain a better understanding of the significance of online interactions. Although they take quite different approaches to their research (Turkle specializes in case studies while Steunkuehler focuses on an ethnographic approach), both researchers have clearly demonstrated that online game worlds are compelling to their players and provide a window into human social behavior. Turkle summarizes a core attraction to online worlds in this quote from her book *Life on Screen*:

> Women and men tell me that the rooms and mazes on MUDs are safer than city streets, virtual sex is safer than sex anywhere, MUD friendships are more intense than real ones, and when things don't work out you can always leave. (244)

As Turkle's subjects state, and Steinkuehler's subjects echo, massively multiplayer online worlds provide a unique kind of social interaction that we find increasingly resonant. The statement above exhibits the complex ties these worlds form deep in our psyche, where our fears and desires mingle in an environment predisposed to transgression, but also packed with emotional significance. It is impossible to clearly describe or fully explain, which is why research into MMOs has been one of the busy areas of game studies scholarship.

These are the varied and complex feelings elicited by online worlds that drew me to experiment with the MMO form in *Hour: 720*. In persistent online environments

people of all cultures are regularly immersing themselves into all sorts of fantastic landscapes populated with a rich array of personalities. Players take up the goals and motivations outlined in the initial fictions of the game, and from there they develop virtual decades of history with the world and its inhabitants.

Hour: 720 seeks to put users into an intense environment with little to no hope of survival. The experience is reduced and concentrated in a variety of ways and the game is formally unconventional. The goal is not to explain or promote a specific set of behaviors; rather, I want to provoke interesting social situations, to force players to make moral and ethical decisions about how they will behave in the game world.

A brief history of online worlds

Most persistent online worlds fall into the category of role-playing games (RPGs). Role-playing games have existed since the 1970s when *Dungeons and Dragons* (1974) was created by Gary Gygax and Dave Arneson. RPGs like the original *Dungeons and Dragons* have become known as "pen-and-paper" RPGs to distinguish them from the many video game RPGs. The early pen-and-paper games created many of the basic RPG genre conventions that have continued into video game development, but the experience of playing a pen-and-paper game remains very different from playing a video game RPG. Even in MMORPGs, where the "group adventuring" aspect is exploited, the feeling of playing a game with others over a computer network is not at all the feeling of playing a game together with friends in the same room. In-person games are spontaneous, collaboratively created narrative events; online, games are scripted by remote developers.

In pen-and-paper gaming, an imagination and consenting fellows is all that is required to modify the game to add a new monster or challenge. In electronic RPG games, changes are arduous to make, and often restricted to content supplied by the developers. These elements change the feel and tone of gameplay significantly.

One of the major aspects of traditional RPGs is a focus on character-development (usually through the accumulation of "experience points" that can be redeemed at intervals to increase a character's attributes) (Filiciak 91). These are games that undo one of the prime assumptions of any game: that a player can win or lose. In role-playing games, the emphasis is put on the *role-playing*. Players create characters who are heroes or villains, sometimes very much like the player and other times completely different. The enjoyment of playing the game comes from building the character (many players create involved histories for their characters, and gaming groups may have evolved entire worlds of lore and mythology).

The original inspiration for the premise of games like *Dungeons and Dragons* can be found in the fantasy novels of the mid-Twentieth Century, especially the 1960s resurgence in popularity of Tolkein's *Lord of the Rings* trilogy. This heritage in fantasy fandom continues to shape RPGs of today. The vast majority of role-playing games are based on fantasy worlds.

The RPG genre was one of the first to make its way to the computer. In 1975 the rules set for the recently released *Dungeons and Dragons* was adapted into a game called, simply, *Dungeon* by Don Daglow. The first MMORPG was created in 1978 by Richard Bartle and Roy Trubshaw and called *Multi-User Dungeon* (MUD). Both MUD and

Dungeon were text-based games, although *Dungeon* incorporated both the graphical map view and a "perspective" view—that is, a graphic that represented the scene in front of the protagonist. As this perspective view grew from the abstract dungeon schematics to finely illustrated still images and then to full-motion animation, the image representation of the environment became the dominant on-screen element. All other informative and textual information has systematically been downplayed in contemporary RPG videogame interfaces.

Single-player RPGs developed in two ways: the home video game console genre and the personal computer genre. The division between these two styles is largely supported by a regional divisions. Although there were early American console video game RPGs, the game console style of RPG was dominated by Japanese developers, and especially the rise of the *Final Fantasy* series, which came to the US in 1990. American RPG fans primarily played on the personal computer, a platform that attracted more Western developers and cultivated a unique style. These games utilized low-resolution graphical worlds and overall incorporated more of a graphical focus.

The American computer-based RPGs like *Ultima* (1981, Richard Garriott), *King's Quest* (1984, Roberta Williams) and Bard's Tale (1985, Michael Cranford) utilized a mostly text-based information display system with an accompanying "perspective" image that illustrated the scene the player's character currently faced. These games also focused on a more stoic sense of the role-playing game and typically offered far more choices to the player in terms of avatar and world complexity.

In the early 1990s the MMORPG really took off. The development of new text-

based systems based on MUD led to noteworthy textual worlds such as *LambdaMOO*, the experimental online world created in 1990 by Xerox-PARC researcher Pavel Curtis. Thanks to the low-impact requirements of the text-based worlds, many of these early experiments are still running and available on the contemporary Internet including *LambdaMOO*. Of course, *LambdaMOO* was not an RPG in the traditional sense because it lacked many of the gameplay systems that are conventional in the RPG genre. Namely, *LambdaMOO* had no mission or quest structure, and characters did not perform tasks in order to earn experience and increase their abilities and attributes. Rather, players made avatars which they did *role-play*, but this play was much more freeform than what we would see in even the most open role-playing games. Still, *LambdaMOO* did offer a fantasy-themed environment and many MOO and MUD players also play online RPGs.

However, as the text-based worlds were growing, so were the graphical worlds. In 1991 America On-Line (AOL) released the first graphical MMORPG, *Neverwinter Nights*. *Neverwinter Nights* was available to AOL subscribers and started out allowing only 50 players to be in the world simultaneously. By the time it was shut down in 1997, the game supported 500 simultaneous players. In 1997 Richard Garriott and Ralph Koster brought the popular *Ultima* series online with *Ultima Online*. By 1998 *Ultima Online* had 100 thousand players, and in 2003 the game served 250 thousand players. The longevity of these games cannot be ignored, and both Koster and Garriott have remained major figures in the MMORPG field.

Since the graphical MMORPG burst onto the scene many have been released. Sony's *Everquest* (1999) was a watershed moment in the coming flood of online games.

Microsoft released *Asheron's Call* in 1999, and the trend continued to gain momentum. Some of the major titles in the MMORPG landscape include: *Dark Age of Camelot* (2001), *Lineage* (2001), *Lineage II* (2004), *Final Fantasy XI* (2002), *Ragnarok Online* (2002), *Star Wars Galaxies* (2003), *Eve Online* (2003) and *World of Warcraft* (2004), which has quickly become one of the most iconic MMORPGs with over 1.5 million subscribers as of 2005. According to numbers from the game's publisher, Blizzard, *World of Warcraft* regularly sees 500 thousand simultaneous players. There has been a steady rise in the number of subscribers to MMORPGs, and according to data tracking website *MMOG Chart* there are currently nearly five million active MMORPG subscribers worldwide.

The conventions of the MMORPG

Across the many games that have formed the history of MMORPGs, a set of conventions has arisen to govern the genre of the MMORPG. As with any genre, conventions arise partly from the inspirations of the developers, partly from what consumers enjoy and partly from what is profitable in the video game market. There are two major categories of MMORPG conventions. Rules govern the working of the game and are typically inherited from the pen-and-paper forerunners of the genre. Formal conventions govern the interaction the user has with the game, the look, feel and working of the interface.

Rules-based conventions

Most genre definitions of MMORPGs begin by defining the RPG element. Typically, the core requirement for a “role-playing” aspect within a game is considered to

be the development of a character, usually by increasing various statistics. Statistics often include attributes such as strength, health, charisma, and intelligence. Players usually raise these statistics by earning "experience points" which are awarded for carrying out different actions in the game world. A certain number of experience points may be gained from defeating a monster, while negotiating with a merchant may earn a different number of experience points.

In addition to the base attributes, RPG characters are often governed by the "skills" they possess. Skills are usually a core component of any RPG, and often they form part of the unique flavor of the game. For example, many RPGs have skills such as "blade" or "conjuration" but a clever developer may choose to augment the game by using skills to suggest behaviors: For example, the bawdy game *Sociolotron* includes a "prostitution" skill, which then opens up a whole career path that generally does not exist in traditional RPGs. Skills are often raised alongside attributes in reward for earning experience points.

Improving an RPG character is often referred to as "leveling." This slang refers to the genre convention of "levels" which are used to represent the accumulated experience of a character. Often levels are awarded on a graduated system: Level two is achieved at 1000 experience points, level three at 2000, level four at 4000, and so on. Thus, as a character "levels up" she must take on increasingly tough opponents and more glorious feats of heroism in order to accumulate enough experience points to progress. Although the mechanic varies somewhat, most RPGs celebrate the achievement of a new character level by allowing the user to make some choices about which attributes and skills are

rewarded. A player may choose to increase the strength attribute and the blade skill of a fighter, for example, or the intelligence and magic skill of a wizard.

This development and statistics management is a major component of all role-playing games. It is the method by which players can create a character finely tuned to their own desires, enacting strengths and weaknesses then playing out the results of their fantasy creation.

While pen-and-paper RPGs require players to become intimate with every aspect of the rules that simulate the fantasy world, in MMORPG video games the computer handles the bulk of the rules interpretation. However, most MMORPG video games are based in traditional rules sets that utilize standardized concepts to simulate the world. These rules range wide and far, from governing the probability of a monster encounter to complex charts for determining the likelihood of missile weapons hits given certain cover and lighting conditions.

However, while the rules for simulation in MMORPGs are often very complex, they differ from other video games, and other single-player RPGs, in one major aspect: MMORPGs never end. At some point the game may be shut down, but the narratives in these worlds are open-ended, and gameplay is not driven by reaching an ultimate goal. The character leveling system provides adequate player-reward and the social interaction provides continually varied gameplay experiences, making it possible for MMORPGs to carry on their never-ending existence. And the longevity of MMORPGs is part of what makes them enjoyable to players. As was discussed in the preceding historical survey, many MMORPGs retain players far longer than other action or single player RPG video

games. In some worlds, players enact multiple generations of the same family, creating epic stories of familial lineage and daring deeds. In some MMORPGs, players are even allowed to participate in governmental functions, further enhancing the player's connection to the fantasy world.

Actual gameplay in an MMORPG is typically structured into three major phases: Socializing is certainly a part of the game, and getting together a group of adventurers is often requisite to complete a quest. “Crafting” is a term that refers to manufacturing items in the game. Often crafting is required to build up a character's equipment either by making new equipment or by making some other commodity, selling it, and buying equipment. Some games even incorporate gathering natural resources to supply your crafting activities, while others do away with crafting altogether, instead expecting players to spend time killing more mundane monsters in order to accumulate wealth and power. This phase of the game is often tedious and is sometimes referred to as “the grind” of the game, meaning that the “Crafting” phase is of gameplay is something that one must get through in order to get to the “fun” parts of the game.

The final phase of MMORPG gameplay is the quest itself. The quest is a component of any RPG, but in MMORPGs quests are handled through a wide variety of systems. Sometimes players are immediately incorporated into the overarching world narrative and quests come from the King or some other character from the game world. In other game, quests are given out by guilds, so a player must first join a group of other players to receive quests. In still other games, quests are merely posted for all to see and anyone to complete, in a quite mercenary fashion.

The quest and how the quest is handled often makes or breaks a game for its players. Developers must balance giving each player the feeling that she is the most important character in the world, yet maintain a cohesive fantasy world simulation. Some games are oriented towards small groups, while others are geared to simulate clashes between huge armies, and the difference in emphasis greatly changes the way players experience the game.

Formal conventions

In addition to the conventions of rules that govern the gameplay, the formal conventions that govern the appearance and player's interaction with the game world are also significant. There have been several different types of MMORPG created over the years, but the various systems have a lot in common, in spite of significant aesthetic differences.

In all MMORPGs, the game must somehow situate the user in a world. This is often done through a combination of images, informative graphics, and textual description. In the early text-based MMORPGs, each location, item and character had a textual description that appeared when a player interacted with that location, item or character. Game developers quickly incorporated graphics, and the “perspective image” became a convention of the genre. This image would show the scene currently in front of the user, such as a forest trail with two goblins. Sometimes the perspective image would be less illustrative, showing an informative, rather than representational, rendering of the location and any objects or characters.

As game graphics have improved, developers have worked to represent the game

environment as accurately as possible in real-time. They have succeeded at creating better and better graphical engines that have facilitated a much more cinematic approach to the game's formal characteristics. Contemporary RPGs, in general, work to maximize the amount of space given to the representative image (which moves in real-time) and create a film-like experience.

But just because players can “see” the game world in real-time doesn't mean that informative graphics have gone by the wayside. Maps and statistics-rendering packages included in many MMORPGs are robust. In worlds as large as the most successful MMORPGs, it is quite necessary to have a good mapping system, and these systems are often rendered in an abstracted form to highlight particular information, rather than the representational style of the game action screen.

Different RPGs experiment with all sorts of information displays, often relying on icons, pop-up menus, and context-sensitive menus. Thus, a player may right-click on a character to talk, or left-click the magic icon in the commands bar and select a new spell they'd like to ready for casting. Statistics like health, magic, stamina, etc. are often displayed as icons or meters with little explicit explanation. Part of learning a new game is the effort required to learn to read the various icons and measures of the game screens.

In addition to the main game action screen, players often have access to several sub-screens. One of these is usually an inventory. What a player's character can carry is often dependent on an encumbrance statistic, derived from strength or some other combination of attributes. Another important area of the MMORPG is the character screen, where players can equip and manage their characters. This screen generally

includes a visual representation of the character as well as all of their statistics: attributes, skills, etc.

In all of these screens, and with the emphasis so often put on statistics and numbers-crunching, MMORPGs take on a formal quality very much like standard computer interfaces. Pull-down menus, check boxes, radio buttons and text entry areas are not uncommon. The worst examples of design interface are overwhelmingly complicated and difficult to manage. The best allow quick and easy access and modification of the information, as with any application.

While these rules-based and formal conventions define the basic characteristics of most MMORPGs, there are certainly examples of aberrant cases. The RPG has not been around for long, the video game for an even shorter period of time, and MMORPGs for only the past 25 years or so. The genre still regularly produces interesting experiments that may one day become conventionalized, but at this point the genre conventions outlined above should provide a good, generalized point of reference for understanding *Hour: 720*.

The experimental game goals of *Hour: 720*

The motivations behind *Hour: 720* are quite different from the mainstream gaming industry, although not so different from most game developers. *Hour: 720* hopes primarily to deliver a compelling experience, one that allows the players to explore themselves and their relationship to the roles they inhabit as they play the game. All

MMORPGs are social, but most seek to provide comfortable refuge from reality. In most MMORPGs, the game is supposed to be challenging on a certain level, but also entertaining. *Hour: 720* problematizes the relationship players are allowed with the game, largely by assigning characters and forcing an abbreviated timeline. The goals of *Hour: 720* go beyond the gameplay itself, indeed beyond the players themselves, and seek to prompt users to consider the roles they play in more detail than most MMORPGs ask.

Re-visioning genre constraints

Far too often in mainstream games, just as in mainstream film, the end product becomes an amalgam of tested conventions, largely because of the immense cost of development and the complexity of the game production process. Large companies pay large teams of people a lot of money to create high-end blockbuster games, and development cycles typically range in the 2-4 year zone for even moderately ambitious projects. The games they create must fit into a landscape of mainstream games within the MMORPG genre: They must look like (preferably better than) other games, and they must offer similar (or more) features. To support the immense cost of development, the games must also have a minimum longevity. There have been virtually no MMORPGs that exist within a constrained time period; rather, most games linger until either their player base dwindles to unsupportable levels (without enough income, companies cannot justify the costs to keep the game running) or the publisher releases a sequel that replaces the previous version.

In one sense, *Hour: 720* is borne from a deep love of games and a desire to see them become the intense, inspiring experiences that gamers all over the world have come

to crave. It seems hopeless for an individual, or even a small group, to compete with the well-funded powerhouses who dominate the game industry. Rather than compete directly, *Hour: 720* decides to turn in an alternate direction. Where conventional wisdom and tradition reign, *Hour: 720* subverts it and proposes an alternative possibility.

Hour: 720 runs in 30-day cycles, preventing players from maintaining a character longer than 30 days (and likely much less time). This abbreviated cycle helps keep the game approachable for new users. Experienced users will still find their knowledge of the game's workings gives them an advantage, but new users will never experience the feeling of entering a party where everyone else already knows each other. *Hour: 720* is not built to engender the "hardcore" kind of gameplay prevalent in mainstream MMORPGs, and which serves to support the aforementioned costs and corporate concerns. The shortened duration of time is an experiment in form that would not be allowed to exist in the mainstream market, although it seems reasonable to suspect that a shorter experience in an MMORPG could be just as compelling as a multi-year experience. In fact, most players will never invest such hours into an MMORPG and unless someone develops a shorter MMORPG experience they may never experience the genre.

Hour: 720 also turns away from the driving force of the game industry: the rage towards cinematic 3D visuals. Rather than focus on graphics, *Hour: 720* puts emphasis on textual description, a type of creative generation that is much more accessible for the player as well as the developer. Thus, players can become creatively involved with textual manipulation and the game structures have an easier time coping with the amount

of information required to effectively build the game world.

However, *Hour: 720* resists the genre conventions of what has come to be known as "interactive fiction" or "IF." IF works utilize the old style of command-line interface to facilitate user interaction. The player is presented with some descriptive text and a cursor blinking at a command prompt. If the user wishes to move east, she types "go east" at the command prompt and presses the enter button. This form of interaction is often uncomfortable for users new to the realm of IF games, and even for well-versed fans of the genre each title presents a bit of a learning curve. Hoping to eliminate some of the first-time awkwardness and create a more culturally relevant interface, *Hour: 720* draws its idiom not from the text-based operating system, but from the world of hypertext and websites. Buttons, icons, and linked text are much more intuitive and provide a much more clear idea of the possibilities inherent in the space than the intimidating command prompt of traditional IF.

Hour: 720 diverges from the conventions of character in a typical RPG. Characters in *Hour: 720* are not built by the player to mirror themselves or represent some kind of idealized or fanciful version of self, as in so many other RPGs. The characters in *Hour: 720* are partially-defined roles for each player to fill. Players bring creativity to the role and their enactment of the role, but they are asked to fit themselves into a cast they may not have normally chosen. This decenters the player-experience and calls into question what the "role-playing" in a role-playing game means, why we participate, and where the boundaries between game and reality are drawn.

To further complicate the traditional notion of character in the RPG genre, *Hour:*

720 does not allow characters to "level up." In *Hour: 720*, the focus is not on building the character, but on helping the character survive in a hostile environment. The role that you are "playing" is problematized: On the one hand, the player assumes the identity of the character, speaking through her avatar's lips, but the player is also the character's guardian, or shepherd, guiding the character through the world. Character development does not come through the raising of character statistics, but through the empathetic connection forged between player and avatar, prompted by the game system, but enacted through player creativity.

Narrative generation

Hour: 720 facilitates and encourages players to contribute to character-based narratives in any way they'd like and construct "game-novels" from their play experiences. The resulting game-novels can be used in a variety of ways by the player. The player may decide to keep the game narrative private, or opt to share their work with the public at large. In choosing to share work, the player-created narratives are made available to the public for further manipulation, download or publication. At any time the game-novels may be hosted on the player's personal website or otherwise shared under a Creative Commons intellectual property license.

This form of narrative generation also runs contrary to genre conventions of MMORPGs. The typical RPG narrative focuses on the protagonist's generally linear progression through a main storyline. There are often small concessions made to player-preference, such as the ability to assign the protagonist a customized name or engage in ancillary side-quests that typically do not affect the overall outcome of the game.

Traditional RPGs frequently make use of branching plots and make claims of narrative innovation. However, these concessions do not change the fact that the traditional RPG exerts a massive amount of control over the story experience as conveyed to the player and provides little opportunity for the player to insert herself into the game's narrative in any meaningful way. MMORPGs necessarily differ from the traditional single-player RPG because they must accommodate many more players. This leads to the perennial problem of how to make a compelling narrative for each player that allows her to feel as if she is "saving the world" (or some similar level of accomplishment).

Hour: 720 puts users in an extreme narrative position and then asks them to either survive or die, in more ways than one. Maintaining the character is certainly a concern and priority, but another concern is the generation of each character's story. Given prompts of character, outside events happening in the game world, and the multiple environmental pressures, players are expected to expand and texturize these events. The player is playing another game in addition to the survival MMORPG—she is playing a popularity game to see which player can make the most engrossing narrative based on gameplay, as judged by the *Hour: 720* audience.

Hour: 720 is a project that presupposes game genres can be tweaked and transcended. Given the flexibility of the digital medium, video games have no cause to solidify into the highly conventionalized genres as we've seen in recent years. The mainstream solidification of game genres is due primarily to financial interests and has little to do with the reality of what interactive entertainment could become. The lack of a

profit-oriented motive allows *Hour: 720* to explore the terrain of genre and convention with an eye steadily fixed on player experience. Only by taking risks with genres and making radical experiments in game design can game authors, artists, developers and designers discover the new forms that will propel the form of the video game to new heights.

Chapter Two:
Welcome to the Disaster Zone

30 days inside *Hour: 720*.

Hour: 720 is a free game, and it runs entirely in one's Web browser. It requires no special software or extra plug-ins, and can be accessed by virtually any computer connected to the Internet. Potential players visit http://www.hour720.com where they find a variety of informative materials describing the game and how it is played. Once they decide to play, a user can register an account and enter the game world immediately. There is no installation or significant delay.

Each game of *Hour: 720* runs for 30 days—720 hours. During that 30 days, players may play several characters, but only one character at a time. Once a character has died, the player can create a new character and continue until the game world comes to an end on day 30. The game is designed to accommodate one thousand players in total, although far fewer will likely be online at any given time. Once a game has ended, a new game begins, so every month the disaster repeats itself for a new audience.

It is also important to note that players are allotted a fixed amount of *action points* in the game. These action points are deducted in varying amounts depending on the action being performed, and they regenerate over real time. Action points serve to limit the amount of time players can spend in the game world. This limitation serves two purposes: Limiting action points allows the server load to be managed in order to keep costs low, and it limits the amount of time any individual player can spend in the game world.

Getting into the game

The player enters *Hour: 720* through its website, located at http://hour720.com. This domain serves as a central point for both the player and spectator's experience of *Hour: 720*. There is a lot to the website, and it will grow as the game develops. It is hoped that users will find the information they need quickly and will see easy access points for digging deeper into the site and game experience.

The website serves both of the game's audiences: the players and the spectators. Players will find easy access to the game interface linked from every page on the website. Whether browsers enter through the main page or through linked content, they will have easy access to sign into the game.

During the running of a game, the site's main page will become an informative resource for spectators to find out what is happening in the game world. This allows spectators to access the overarching world narrative, which is the story of the government, media and public responses to the outbreak of zombies attacking the island.

As New Hampton falls into chaos, mysteries are revealed and factions take sides in the official and unofficial responses to the emergency. To an extent, this overarching narrative is more accessible to spectators, or to players outside the game environment (when they visit the main page or tune content syndicated through RSS feeds).

After visiting the *Hour: 720* website, browsers can sign up to play the game. The number of players is limited to 1000, which is a relatively arbitrary limit attractive for two reasons: As a developer, it seems like overcrowding the world would ruin the ambiance of the game. And 1000 is the arbitrary number required to make a multiplayer game environment “massive” in most genre definitions. So while it is stated that 1000 is the target number for players, that figure should be taken with a grain of salt. If need be, the limit can be lowered in order to not overburden the server, and although that may increase the wait required to get into a game, it shouldn't negatively affect the game experience.

Once the player has signed into the game, she is presented with a user/character management screen. This is the main hub for the user, and it lists the user's past characters and games as well as the active character. Users may only play one character at a time, but once a character's game is over, the player can generate a new character. In this screen the player can also configure any personal information they want to make public, edit and organize stories, chapters and anthologies of their character experiences in the game, and generally perform account maintenance.

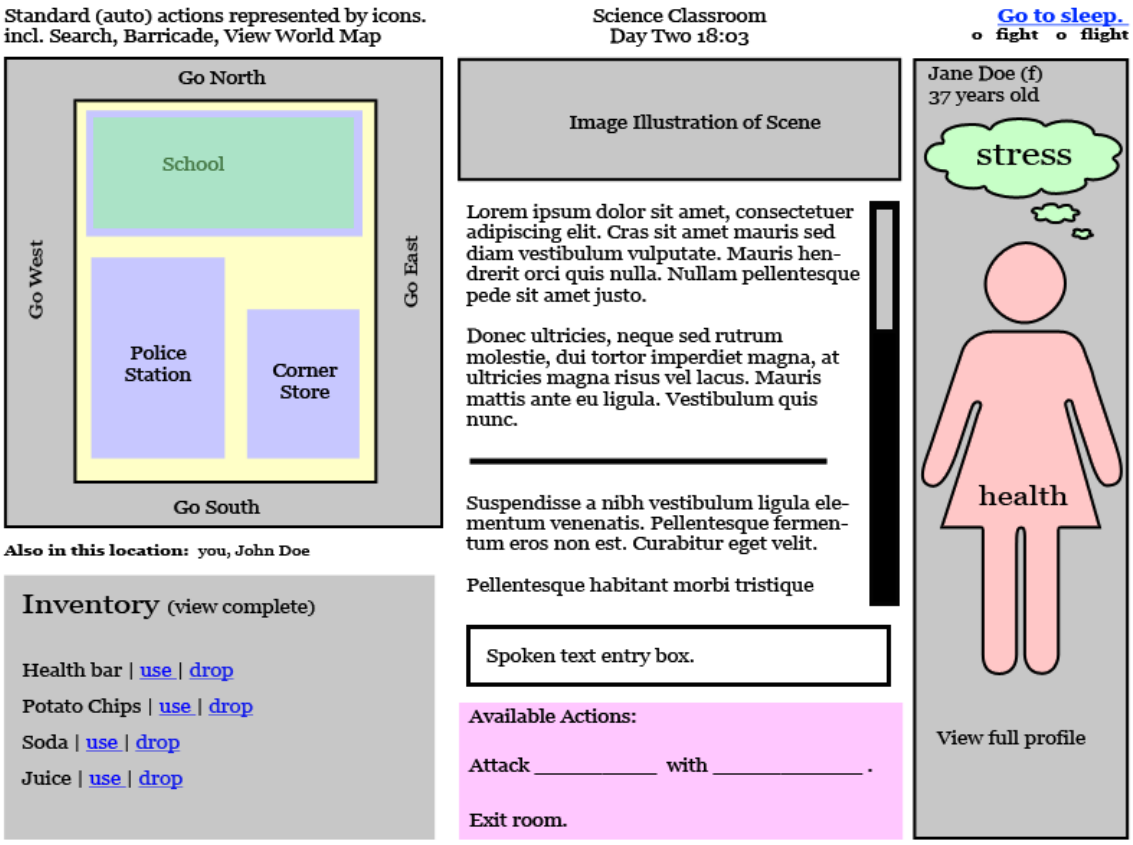

Figure 1: Wireframe diagram of *Hour: 720* game interface.

From the user/character management screen, the player progresses to the game screen proper. This is the “window” in which the player plays, and it provides the player's view into the world of New Hampton. The game screen is designed in the style of traditional RPG interfaces. There is a “map” area, and clicking on the map moves the user from location to location. An abbreviated inventory shows the most immediately useful items in the character's possession, and vital character statistics are shown in the avatar area. In the center of the screen is the text and actions area. This is where the textual descriptions of the game world show up, and below these descriptions are the options for performing actions in the game world ranging from speaking to attacking.

An island in crisis

Hour: 720 puts players into the role of survivors on the island of New Hampton. The focus of the gameplay is on the individual character experience; however, all of the player characters inhabit the same world, and there is an overarching world narrative. This larger plot tells the story of the actions taken by individuals off the island which directly affect the player characters.

After the outbreak occurs, panic and mayhem engulf the island. As it becomes apparent that the zombie outbreak cannot be contained by local law enforcement, there is a mass movement to the only bridge that connects the island to the mainland. The bridge is congested with vehicles and people until traffic is at a stand-still, and the oncoming hordes of undead cause a complete catastrophe at this major bottleneck. Eventually, the regular Army seal off the perimeter of the island, quarantining the survivors and victims on New Hampton.

This catastrophe gets news coverage as various parties become involved. Throughout the 30 days of the game, players will receive messages from the armed forces, various news agencies, and even the President of the United States. These messages are conveyed through radio broadcasts and pamphlets or fliers distributed around the game world. In each message, players are given information about the larger situation and recommendations on the best course of action. It is up to the player to reconcile these sometimes conflicting messages and interpret the rhetoric to discover the true motives behind the public statements. Some officials have agendas that go beyond saving lives or fulfilling their civic obligations.

After 15 days, rescue crews begin to land on the island, hoping to find any remaining survivors (player characters) and assist them off the island. These groups of rescuers come from several backgrounds: Some work for official government organizations, while others work for local civic and private search and rescue groups. Each rescue organization has its own agenda as well. A local rescue crew, for example, may be concerned with finding a fellow member's family, while a regular Army unit may be tasked with gathering information about the outbreak or finding a specific item or person on the island of interest to government officials.

All of these elements form what is considered the "overarching world narrative," which expands the story beyond the individual player character experience. These narrative elements will be woven into virtually everything the player character experiences. The tension between conflicting information and groups of rescuers at odds with each other provides a dramatic puzzle which players must figure out in order to know the best course of action.

Having described the mechanics of getting into the game and managing their participation in the game world, as well as the overarching narrative that bonds the various player experiences together, it is apparent how little a summary of game mechanics and story can tell of the actual experience. In order to understand what it is like to play the game, the following section details the experience of a player in *Hour: 720.*

A summary of player experience in *Hour: 720*

To better understand the details that follow, it's helpful to follow an imaginary player, Jane, through a 30-day period. Let's assume Jane registers at Hour720.com on day one of the game, ready to enjoy the full experience, and continues through in the spirit of the game until the game ends on day 30.

Day One Jane signs up to play the game. She generates a character called Billie Timinsky, a 19 year old unemployed male. Jane doesn't notice that she can customize Billie's background story and jumps into the game world where she finds herself alone in a school.

Playing as Billie, Jane explores a few rooms in the school and acquires a handful of snack items from discarded lunches. These are useful for regaining health. She also finds a broken desk and makes a club from one of its broken legs.

Jane runs out of action points unexpectedly and, realizing she'll have to wait for her action points to regenerate, she closes her Web browser.

Day Two Jane returns to the site to find that her character, Billie, is almost dead, having been attacked by numerous zombies. His mental health icon is also very nasty looking because he has not slept in nearly 24 hours. Jane makes a mental note to be sure to set her character to “sleep” before leaving the game.

Jane eats all of her health bars and decides to leave the school. She finds that Billie is surrounded by a horde of zombies that have gathered outside the school. Jane tries to get Billie to flee, but there are too many zombies. Billie succumbs to the horde and is completely devoured.

Jane is returned to the character management screen. She is invited to

fill in an obituary for Billie that will display in the Hour720.com news coverage of the game world. Jane fills in the form, jokingly referring to his inability to ever get out of school. Jane is also offered the immediate chance to view and edit Billie's game narrative. Unsure what exactly that means, she clicks the “skip this step” button and is returned to the character listing.

There are two entries in her character list: Billie Timinsky is shown as deceased and there is a link next to his name that says “edit.” The second option is to generate a new character. Jane decides to generate a new character and is given Tamar Kier, a 22 year old female EMT. She enters Tamar's game, but has very view action points left. Tamar successfully secures the hospital room in which she is located and tries to get some sleep.

Day Three Jane checks in on Tamar eagerly because after telling her friend about the game she thought of some excellent ideas for Tamar's history. This time, before clicking on the “enter game” button next to Tamar Kier's name, Jane clicks the “edit” button to edit Tamar's backstory. Armed with a strong sense of character, Jane plays Tamar and is happy to find other players present in the hospital.

Chatting with other players reveals a lot about the game world. There

are a couple of different smaller groups, and some player characters never seem to move. Some characters only play in the hospital because they have some kind of life-threatening disease and need regular help. As long as enough characters remain to maintain the barricades, the zombies remain at bay.

Day Four Jane spends another day playing Tamar, chatting with people at the hospital. There are so many it is difficult to make out all of the chatter, and there are no weapons or health to speak of. She and a group of others decide to try to reach the ferry off the island. The latest radio loops say the ferry is still running.

Tamar and two other player characters exit the hospital and move north one block where they enter an abandoned restaurant. There are a couple of zombies inside, but the group is able to defeat them.

Day Five Jane didn't think to make a time to meet again with the others in her group, and when she logs in during her afternoon break at work the other two are no longer present in the restaurant. A note has been left that says they are moving north one block per day.

Jane moves north and finds three buildings: a church, a school, and another house. Jane enters the church and explores it, making sure to secure rooms as she goes. She only encounters one zombie and

manages to run away from it. She enters the house next and finds one of her former companions sleeping there. Before logging off, she writes a note that says she'll follow north and look for notes.

Day Six Being the weekend, Jane doesn't pay much attention to the computer.

Day Seven Being the weekend, Jane doesn't pay much attention to the computer.

Day Eight Jane returns to the game world and finds that Tamar has been attacked and killed by another player. Ana Magana has apparently attacked her in her sleep. And to top it off, Ana Magana is a priest! How could this be? Jane ponders for a moment, then another question creeps into her imagination: If Tamar is dead, then why is the game still going?

Jane realizes that Tamar has been resurrected as a zombie. Ana Magana must have been infected, which probably was why she (or her disgruntled player) attacked Tamar. As she plays Tamar, Jane notices just how different humans and zombies are. Alert messages constantly remind her of how hungry she is. She notices that her health is decreasing as she ponders what to do. She decides to continue north.

Zombie Tamar crosses some suburban park areas and enters a strip mall coffee shop. Inside are two sleeping player characters, not her former companions, but easy pickings. One of them gets away with no damage, but the second one, a female felon named Wilhelmina

Hattier, is hers to feast upon. As Tamar feeds, her health returns, although her mental state never improves.

Having used up her action points, Jane logs off without bothering to secure the location.

Day Nine Jane listens to the *Hour: 720* radio feeds in iTunes on her work computer and hears a report about possible rescue teams coming within the week. She logs into the game from her work computer and checks on zombie Tamar. Tamar is hungry, and nobody else is in the building. She wanders the world, finding plenty of other zombies, but unable to communicate with any of them. Quickly, Jane is called away by a co-worker and just closes the Web browser.

When she returns to the game later that night, Tamar is still standing in the street. There are many zombies around her, but none of them are bothering here. She is very weak from starvation, and Jane sets out to find another unsuspecting player character. She finds some secured buildings she cannot break into, and an open house that is empty. She hides in a bedroom and hopes to catch someone tomorrow.

Day 10 Jane logs in and just as she had hoped, she finds a couple more player characters sleeping in the house. Zombie Tamar attacks the first one, but she is so weakened from hunger that he kills her in defense (but

not before she got some good scratches and bites in).

Zombie Tamar's game is over, and Jane spends the rest of the evening writing her game obituary and toying with the game story editor to make Tamar's narrative as compelling as possible. Tickled with herself, she saves out a PDF version of the story to show her friends.

Day 11 After seeing the story, a few of Jane's friends want to play. They also login to the site, but the problem is that only two of the four characters begin the game in the same location: the Grand Hotel Ballroom.

The Ballroom is being used by the city as an emergency gathering point, although help has so far failed to arrive. Jane is spawned in the Ballroom as Duane Hufnagel, a 42 year old laborer. He is very strong and she has become used to the game enough to be impressed with his statistics and happy she got him as a character. Jane's friend, Joe, is playing Genia Windauer, a 55 year old reporter. Jane and Joe and their two friends open up an instant messaging application to coordinate their efforts and figure out that the other two friends have spawned very far away from the Grand Hotel, one in the shoreside entertainment district and another in the rural area in the center of the island. Jane reminds them to get into a building and secure the

location before they run out of action points, and the two begin their trek into the city.

Jane and Joe enjoy chatting with other characters and prepare themselves to embark on a journey to find their friends. Joe notices that he has a button available on his screen that Jane has never seen: “file report.” He clicks it and is presented with a text box to enter a headline and article. Jane realizes that must be how those news stories end up on the front page of the Hour720.com website and encourages him to do a story. She gives him some quotes to use from her character and Joe writes a handful of quick lines in his best faux reportage style.

Day 12 The next day Joe's report is on the main page of the website and before Jane shows up at work he's shown it around to half the office. Jane and Joe jump into the game during a break at work and Jane is surprised to find that Joe isn't really so excited about leaving the hotel ballroom. He wants to write at least one more story about the ballroom and since he only gets to write one per day, he wants to stay another day.

Jane agrees, but only because she's read a page on the Hour720.com website that mentioned the invisible friends system that increases

your abilities based on your friends. If a friend is present when performing an action, then the character is much more likely to succeed, making it much easier to survive. Jane decides it is not a bad idea to keep learning more about survival techniques and build up her friends list, even though she still didn't quite understand how, or if, she could impact her friends list.

Day 13 The next day at home, Joe and Jane login together again, and this time they are able to coordinate easily their actions and move together through the game. They make it up to where one of Jane's friends had slept, but find that their friend's character has already been turned into a zombie.

Joe is excited to see action in the game, but hesitant to participate. Fortunately, one of his interviewees at the ballroom had traded him a shotgun for good placement in Joe's latest article, and combined with the social bonus of a traveling companion, the zombies prove to be relatively easy pickings.

Jane decides to do the honorable thing and kill her friends zombified character herself. In the scuffle it is unclear to Joe whether or not Jane has been bitten by the zombie and possibly infected. Jane is adamant that she was not bitten, at least not that she knows of, and the two

camp out.

Day 14 After Jane's other two friends make it clear that they don't particularly care for the *Hour: 720* experience, Jane and Joe decide to try to reach the ferry that Jane had heard about several days ago.

They begin their travels, encountering the occasional zombie and camping in abandoned buildings. Joe writes a humorous feature about the history behind the Jenelle J. Schellenberg Monument, which they pass.

Day 15 Jane and Joe continue their travels to the ferry. Joe writes a road story article for the Hour720.com news pages.

Day 16 In a scuffle with a zombie, Jane is convinced that Joe's character has been infected. Unsure how to confirm the infection, she keeps track of his health and brings it up. Joe says he has noticed his health dropping regularly, but he doesn't think he's infected.

Day 17 Joe's character, Genia Windauer is definitely infected. Partway through the day's game session, she dies and is resurrected as a zombie. Jane is not caught off-guard and manages to kill the zombified Genia Windauer. There is no other reporter present to file an article, but Joe spends the rest of the day tinkering with his character's obituary.

Day 18 Jane and Joe are surprised to see that Joe has respawned as Samuel

Stotts, a soldier with the National Guard sent to rescue the family of a US congressman. Joe is able to make his way through dangerous territory thanks to his far superior equipment.

Day 19 After a couple days of travel, Jane and Joe have managed to meet up in the game. Together they will look for the lost family.

Day 20 Seeking out the family, Jane and Joe encounter a large group of zombies and Jane is infected. She is sure she's infected this time around and Joe can notice it as well. Joe comments that if he gets infected again he wants to play out the zombie role—especially if he can keep his machine gun. Jane decides she will be the better zombie and promises not to attack him when she turns.

Day 21 Jane and Joe determine that the family is not in their house, nor are they located in the Grand Hotel Ballroom (where Joe had to investigate alone because they both decided that the crowd would kill an infected player character immediately).

Hiding in the back room of a fast food restaurant, Jane undergoes the transformation into zombie and plans to see how long she can last.

Day 22 The two make their way towards another popular gathering point, the Chadwick R. Gustafson Memorial Hospital, where Jane had spawned in her first game. Along the way Jane's character is overcome with hunger and unable to resist the urge to feed. Through no action of her

own, the zombified corpse of Duane Hufnagel attacks Joe's character, who retaliates and easily kills the weakened zombie.

Day 23 Jane makes another character, and finds herself playing the role of Janette Beezly, a special operative working for a government contractor who has been hired to contain the outbreak to the island of New Hampton. Janette is tasked with insuring the destruction of a particular filing cabinet located in the offices of another government contracted research firm.

Although she does not rendezvous with Joe in this session, Jane is a bit sad to hear that he has found the dead corpse of the congressman's wife. He decides to help Jane on her mission.

Day 24 Jane and Joe manage to meet at a point near their ultimate destination. The research facility is located in a suburban area of the island, amidst parks and strip malls. The two heavily armed characters have little trouble with zombies in the area and manage to complete Jane's task.

Day 25 The two head back to their respective mission completion points. Joe must return to his search and rescue unit in order to escape back to the mainland, and Jane must return to her insertion point and signal an offshore contact who will discreetly pick her character up.

Day 26 On returning to his team's landing point, Joe discovers that the group has mostly disbanded and many have been killed. There are no boats

waiting to take his character back to the mainland. Instant messaging to Jane, he discovers where she is heading and decides to hitch a ride with her people.

Day 27 Jane waits for Joe to show up before signaling her boat. After calling for her ride, the boat arrives and picks her up. There is no option for Joe's character to accompany her. She is forced to leave him behind.

When Jane's character departs the island she is presented with an epilogue that states she has successfully completed her mission and is an asset to her country. She is not given the chance to make a new character, so she spends some time touching up the details of her adventures and making the beach departure scene as sad as possible.

Day 28 Jane finds she still cannot make a new character. Joe can login to his character and is making his way back to the Grand Hotel Ballroom where he hopes he can wait for rescue. The radio reports that rescue missions to the island have been ordered to stop and any remaining rescue personnel on the island are supposed to get off the island within 48 hours.

Day 29 Joe leaves to look for a boat with a group of players from the hotel, but most of them don't make it far. There tends to be an ideal group size, Joe has noticed, and they were too many. With a group of four, they make their way closer to the shore. Radio reports reiterate that all

humans left alive on the island of New Hampton must evacuate by any means. The rumor is that the government is preparing to destroy the island.

Day 30 Joe logs in to play at work and Jane watches over his shoulder. She is upset that she cannot play the last day of the game, so they are sitting together. Based on the time the game started, the two have calculated when the end will come and are present to witness it. The radio stations have been taken over by the government and broadcast the same message: The island has been cleared of inhabitants for the most part, and any remaining residents would be considered collateral damage. This was the greatest human disaster of all-time, and for the good of all mankind these people must be sacrificed.

Jane and Joe wince at the broadcasts, and sure enough, at the final hour, something happens. It's unclear whether it was a nuclear bomb or some other kind of explosive or experimental device of mass destruction, but it's very obvious that where New Hampton once stood was now a desolate wasteland.

The two sat staring at the final text for awhile, then chattered about the finish and what it would mean. Within a few minutes the character management interface updated to offer them the chance to join the

next game starting at Hour: 0.

The above prose details one of many potential experiences in *Hour: 720*. And these first 30 days might only be the prelude to the real fun, which for some will undoubtedly center on creating their game novels and sharing stories with their friends. Our imaginary players can, on day 31, create a textual narrative of their adventures in the game that is able to be distributed to their friends, printed in book format, and generally shared among any interested parties. The player can choose to make this available to others to read, as well, further extending the experience of the game world.

Chapter Three:
Raising the Dead

The technical structures of *Hour:720*.

The genre of MMORPGs runs the gamut from high end, extensively graphic production values to their roots in independently-created, text-based experiences. Indeed, game design at large has experienced a tension between the increasing production values of the mainstream gaming industry and the desire of some gamers to experience more unique, creator-driven work. Often small development groups feel they are excluded from the full potential market for their game due to retailer/distributor agreements, lack of mainstream gaming coverage, and cost of development.

In part because the game developers know the frustration of getting one's independent vision out to the public, and in part because it makes good fiscal sense for mainstream game publishers, a wide variety of game development tools have become available. Many of these tools are highly accessible, available at low cost and requiring little technical knowledge. Development and modification tool sets often come packaged

with mainstream games, too, such as Valve Software's *Half-Life 2*, which was awarded PC Game of the Year in 2004 by nearly 40 different groups and publications (Valve).

However, many of these game development tools have two major drawbacks: They vary greatly in the amount of creative freedom one has to enact a unique game vision, and they are typically built around very strict formal genre conventions (such as *Adventure Game Creator* or *FPS Creator* in which the genre constraints are advertised in the name of the software). With *Hour: 720*, we wanted to create a game that turned away from many of the existing genre conventions of the MMORPG to create a new kind of role-playing social experience. *Hour: 720* focuses on text and icons rather than a graphical environment. It is meant to be, in large part, a reading experience. But the genre of "interactive fiction" felt constraining because it has become highly conventionalized. Most interactive fiction development systems (of which there are several) do not allow for a high level of deviation, and virtually all of them lock developers into a command-line interface for user-interaction. While *Hour: 720* is highly textual, the command-line interface is not intuitive enough and seems archaic compared to the many ways we experience textuality in the online world. Furthermore, many of the existing independent development tools also require some kind of plugin, helper application, or viewer to experience the resulting game. It was crucial that *Hour: 720* be playable on virtually all computers connected to the World Wide Web. The fewer software requirements necessary to engage with the game, the broader the audience, and *Hour: 720* relies on access to as broad of an audience as possible.

It is also worth noting that virtually none of the aforementioned development

tools allow users to create massively-multiplayer online (MMO) games—that is, games which are built for a huge number of users to play simultaneously. There are some solutions for independent developers, such as the *Caneva* system, but they rely on both highly conventionalized game engines and the leasing of servers. Although rates are established to allow small businesses and ambitious individuals decent access to powerful tools, this was also not "right" for *Hour: 720*. The textual game has the luxury of requiring far less bandwidth and server space.

There was also a need for a development system that could accommodate electronic collaboration. Asset development, in a textual game, should not be difficult to share instantly and robust collaborative text creation tools exist online. The basic text and media content management systems used by many different websites were evidence that online tools could accommodate *Hour: 720*'s needs.

The solution to developing *Hour: 720* as a game was to build it using technologies standard to general web development rather than tools targeted to developing games within a particular genre. Standard web technologies allow *Hour: 720* the flexibility of a website to target many different end-user platforms (Windows, Mac, Linux/Unix, as well as handheld and/or mobile devices). Those same technologies provide for robust development tools that have helped keep the various contributors working together. It has been very important for contributors to be able to add their efforts to the game without requiring additional encoding or adaptation on my part, the lone software developer on the project. Finally, by using basic Web design methods to create and run *Hour: 720*, it was possible to run the project on a standard web hosting

account, which is highly affordable for the independent developer. For far less than the cost of a home phone line each month *Hour: 720* can support about a thousand players.

Having settled on the conceptual design decision of using regular Web technologies to create the game, the project then faced a serious question: Which Web technologies? Closing the door on traditional independent game development tools opened another onto a vast landscape of choices yet to be made.

Technical decisions underlying *Hour: 720*

There are many options available for various types of website development, but it was quickly decided to work with the very popular combination of PHP for server-side programming, mySQL for database management, and Javascript for client-side programming. These technologies can be used with any web server platform, but *Hour: 720* was targeted to the Apache web server model.

In part, these decisions were dictated by the availability of technologies from common web hosting services. It is important for *Hour: 720* to run at low cost and is ideal for it to be easily installed on another web server without requiring much special configuration. Although other programming languages such as Perl or Ruby could have been used, PHP supports the necessary functions of the game and facilitates and object oriented program structure that allows the *Hour: 720* software to remain flexible and expandable.

The common language of web browsers tends to be HTML. *Hour: 720* complies with XHTML and CSS guidelines, which are the most common standards online. This

allows *Hour: 720* to be viewed by all contemporary web browser software. The flexibility of the server-side response also allows for the game to be easily adapted to mobile platforms, and in the future *Hour: 720* could be played on cellular phones and PDAs as well as common personal entertainment devices such as Sony's PlayStation Portable or the Nintendo DS. In this way, *Hour: 720* is able to cohabit with both the Web at large and the mainstream gaming audience.

In order to create a dynamic in-browser multiplayer game, it is important for the user to receive updates about the game world. In traditional Web-based applications, the page must be refreshed in order for the server to send information to the client (user). There are two aspects of this type of browser-refreshing interaction that are major drawbacks: First, the entire page reloads and often creates an unsightly “blink” or, worse, slow loads that make it impossible to view the information that is not being updated. Second, if the server were to return a complete web page each time the game updated a small piece of information, the bandwidth overhead would quickly become prohibitive.

By using the Asynchronous Javascript and XML (AJAX) method of dynamically updating small pieces of a page without refreshing the entire page, *Hour: 720* is able to bring a level of dynamic interactivity that was heretofore impossible in the Web browser. To facilitate AJAX transmissions from the player's computer to the game server, Paul Visco's *Surebert* library was used. *Surebert* allows easy integration of AJAX communications compliant with all major browser types and is more than sufficient for the needs of *Hour: 720*.

Structural overview of *Hour: 720*

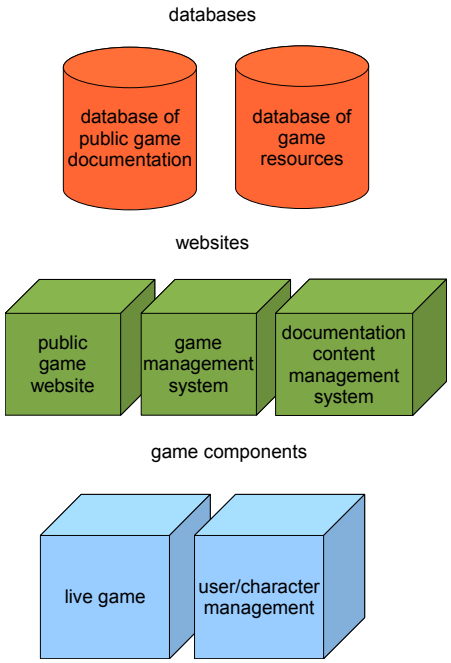

Figure 2: Major components of *Hour: 720*

Hour: 720 is comprised of several major elements (fig. 2). The most publicly visible element is referred to as the public game website. The public game website is an informational resource, and advertising tool to entice new players, and the entry point for actually playing. This is the public face of *Hour: 720*. The next major public component is the actual live game site, which is a collection of two nodes, the user/character management screen and the live game screen. The last Web component is the game "back-end" or developer administration site. This administration site is used to create all of the game elements in *Hour: 720* and allows the developers to create and destroy game

worlds with a variety of settings. There are two databases used in the project. The first is the public documentation database, which contains all of the information presented on the public game website. The second database is the actual *Hour: 720* game database which contains the game creation resources and the current state for the live game.

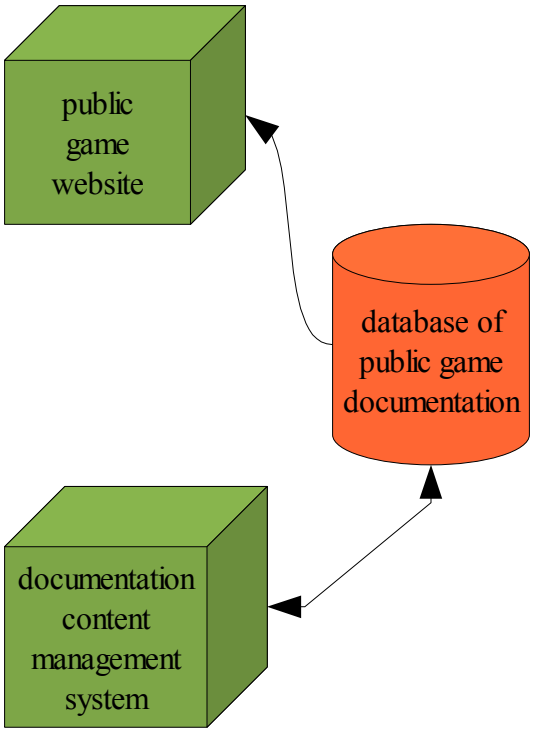

Figure 3: The structure of the public game website.

The public game website for *Hour: 720* uses an existing content management software called *Textpattern* to administrate the information presented on the site. *Textpattern* is convenient to use and allows the necessary code to be inserted to facilitate customized components that integrate the public website with the live game (fig. 3). Using a content management system for this information is necessary to provide easy updates to the information contained on the site, and to further be able to distribute duties

in managing the live game and player community.

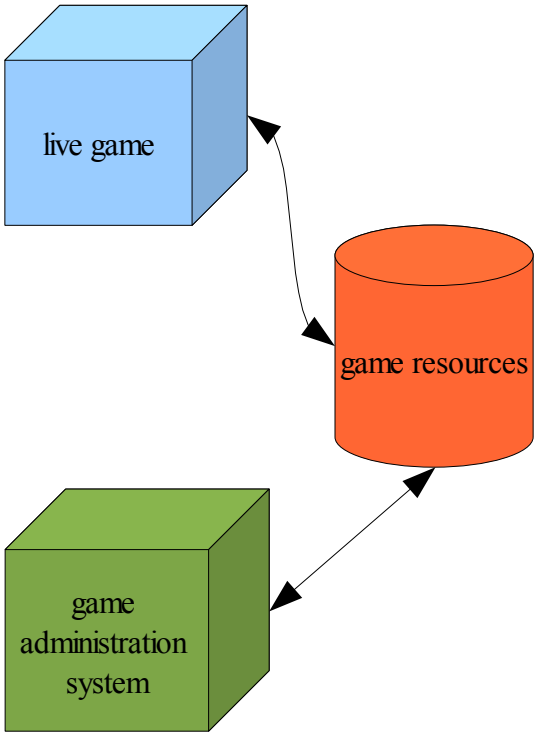

Figure 4: Basic structure of the game administration system.

The back-end administration website, on the other hand, is meant to be used by only game developers, but in spirit it is very similar to the tools included in *Textpattern* (fig. 4). The *Hour: 720* game management system allows developers to modify virtually every aspect of the game and game world. Skills and jobs can be manipulated, settings can be modified, and new locations and items can easily be created and edited. In addition, the basic building blocks of the *Hour: 720* narrative can be modified via the administration interface and tests can be run to preview the effects of these changes.

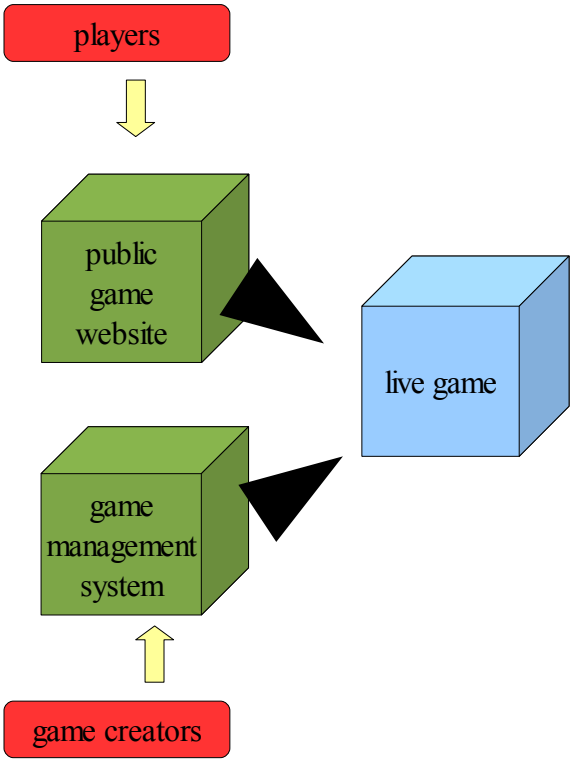

Figure 5: User movement through websites to live game.

Both of these websites impact the live game, albeit in quite different ways. Game creators may login to the game management system and modify elements that players will then find in the game world or in the gameplay. The general player will login via the public game website and then proceed to the live game, where they will affect the gworld by enacting different actions. Figure 5, above, illustrates the flow of players from the game websites to the live game itself, and their impact on the game world follows this same path.

Upon logging into the game, users are routed to the appropriate location based on their user level. Administrators are shuttled to the administration interface, while general players are presented immediately with the live game. The game engine has several major classes that form the main components and handle all information processing. The admin class handles all user login and permissions functions. It also handles all update and modification functions when editing game resources in the administration interface. The gameGen class generates a game world instance. It handles creating the map and placing buildings, rooms and items into the world. The character class handles all character-based functions, which is chiefly limited to character generation. The eventParser class handles all of the information processing during the live game and communicates between the game server and the player's computer. The zombie class is used to manage zombie behavior, creation and actions. Finally, the *Surebert* class is used to facilitate the AJAX communication between the player's computer and the game server as discussed earlier. *Surebert* is the only class not written by Shawn Rider.

To better visualize the flow of information through the game system, it is useful to visualize the game engine as shown in Figure 6. The green arrows represent the path through the system taken by a player. The grey arrows represent the path taken through the system by an administrator engaged in administrative activities. The blue arrows represent program loops that are called and repeat until various environmental conditions are met. The large, dashed green arrow represents the connection between the administration system and the live game.

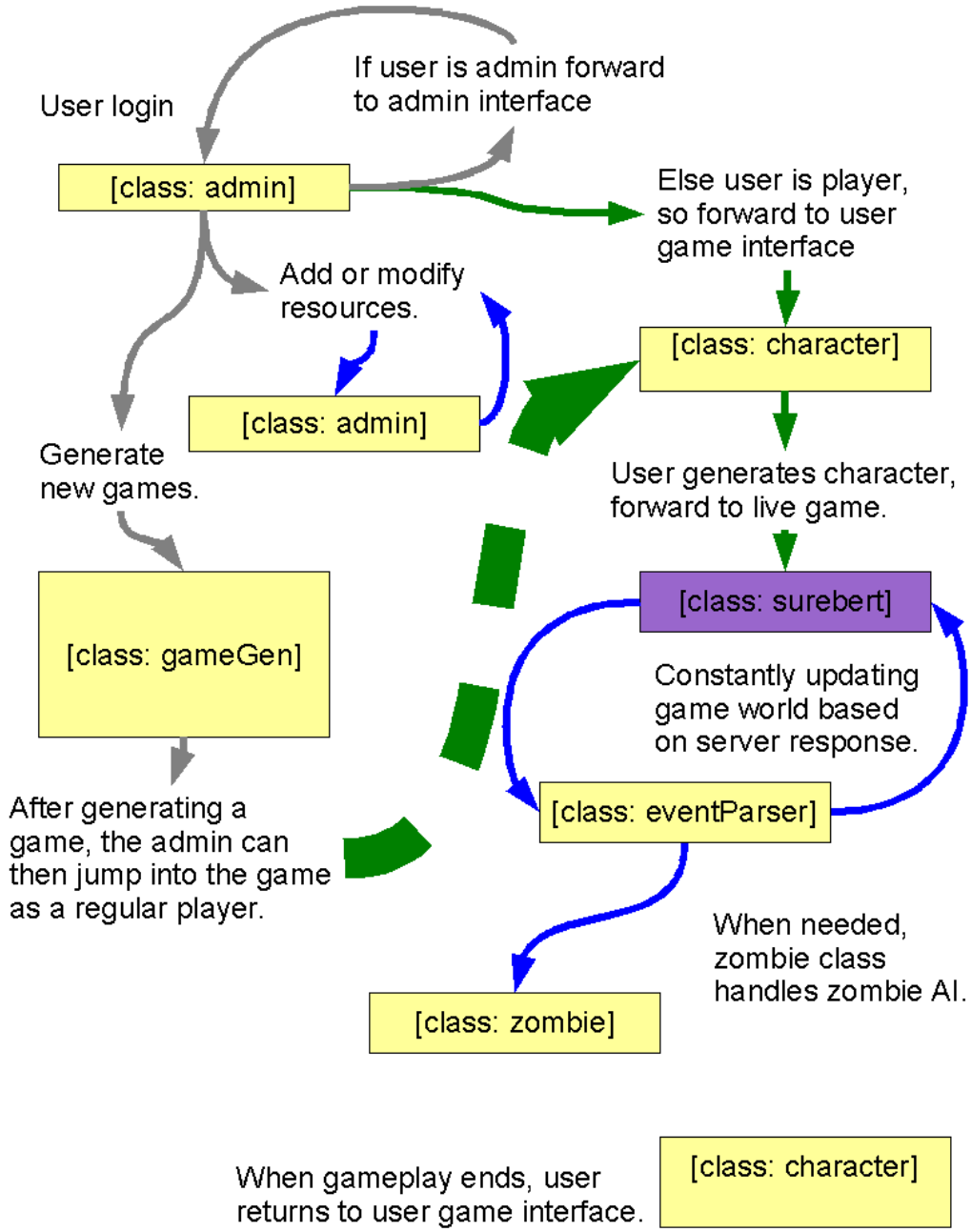

Figure 6: Hour: 720 game engine code flow.

Of special interest is the zombie class, which handles all zombie-related activities, including the zombie AI. Zombies in *Hour: 720* are the same as player characters, except

with degraded dexterity and health. Zombies are also continually nagged by a need to eat. The zombie AI uses environmental awareness (zombies know as much about the environment as player characters do) and hunger to govern zombie behaviors. The environmental variables used throughout the game are primarily cover, light and noise. Cover indicates how easy it would be for a player character to hide from a zombie. Light indicates the level of visibility. Finally, the noise variable indicates how attractive a location is to zombies. For example, if players perform an action or chat in a location, they increase the noise in that location, and more zombies will come to find a meal. Once a zombie has located a target, he will continue to pursue that food source until he is either destroyed or the target successfully escapes. As zombies get hungry, they are more tenacious in their pursuit of player characters and more reckless in their attacks. After a zombie eats, it might pass up a difficult chase because it is still in a digestive haze. The zombie world is free from complex decisions about "what to do." Zombies eat and look for food. There is not much "intelligence" to their AI, but that is entirely fitting with the character of the zombie.

Beta testing the dead

Given the scope and ambition of *Hour: 720*, it was determined that a successful beta test would be sufficient for the purposes of this first phase of research. Beta testing is an essential part of game development because it allows for creators to both work out kinks and bugs in the system as well as react to feedback from testers to tweak the game dynamics and system in order to provide the desired experience. *Hour: 720* is designed to support a large number of users, but, as stated, this number should be seen as a

hypothetical limit. Although it is believed that the game will support about a thousand players, testing has been conducted with about a dozen players. When the next phase of public beta testing begins, we hope to reach a number closer to the thousand player limit, but for now the dozen beta testers have provided excellent feedback.

At its maximum, we've had six players simultaneously playing in *Hour: 720* roaming as a pack, taking on zombies and looking for supplies. Many game interface problems have been identified, and it's clear that the visual layout of the game must change before the final version. Fortunately, given the design parameters of web development, this kind of re-skinning of the interface is easy. In fact, it is not hard to imagine a user-customizable interface in the future, given the ease with which the visual aspects can be shifted.

One thing that becomes clear early in the gameplay is that the more people in the game, the more zombies are attracted to them. Hiding out in a house becomes a good way to stay safe, but it tends to attract zombies to such a degree that getting out of the house can be a very difficult endeavor. Players must literally “run” by clicking fast on navigational points to make it out of the zombie horde and into a more safe zone. After a few of these play experiences, gameplay mechanics make smaller better for overall survival than large groups due to the amount of attraction multiple users create for zombies. Players then began to splinter into smaller groups, which caused the inevitable “team-picking” that *Hour: 720* puts at the forefront of gameplay. Each player must ask herself, “What do I gain, and what do I lose, by helping this person?”

The beta test ran for a full 30 days. Most of the play records show that people

would somehow coordinate playing together online and then meet up in the game. It is likely that users utilized an additional form of communication (another instant messaging client) outside the game as they played. This is a common situation in other online games and there are even some third-party retail solutions that provide voice and/or video chat in games that do not support it. The in-game chat is intended to be "in-character" so it is acceptable for players to go outside the game to facilitate "out-of-character" conversation, such as where they are located in the game world or other maintenance socialization.

Augmenting the game with outside software is also acceptable during the beta phase because not everything in the game was finished during the first wave of beta tests. These tests focused on the mechanics of the game world: Interacting with objects and locations, interacting with zombies and other player characters, and interacting with the Web server to gauge the load incurred to support several players online simultaneously. In these regards, the tests were very successful. The game finished out properly after 30 days and would no longer allow users to play. The game system tracked itself properly throughout the play, distributing narrative content such as radio broadcasts during the proper real-time window.

User response was generally good. There are several strategies players can employ for getting around in the world, and discovering the parameters of the world generally kept players involved for a solid gameplay session. However, after that, it became more of a chore for players to return to the game. When questioned, beta testers overwhelmingly cited the general lack of an overarching narrative. While users enjoyed

the moment-to-moment exploration of the world, they wanted to feel more involved with the larger events. This lack of finished narrative content proved to be a significant issue with testing. Testers just didn't feel motivated to return to the game, and only did so when groups were arranged to play together. Social interaction was a major factor in getting players to immerse themselves into the game world, which also points to a need for more people in general to be allowed access to the game. However, users seemed to play in groups of two to six, depending on how many managed to find each other in the game world. There were never more than twelve players logged-in at any one time, and that only happened once. Even a small two or three player group provided more compelling and entertaining experiences.

Although this quick loss of interest in the game was disappointing, it was anticipated. It's clear that *Hour: 720* trades on experiences and the sharing of those experiences with others. As it is now, the game world consists mainly of a system through which my partner and I can deliver several different narratives, but not all of those narratives are there. The radio broadcasts are written for the first several days, but are in-progress after that. At its core, this generation of narrative components is a massive data entry job which is ongoing throughout the development process and crucial to the full realization of the prject. Currently there are a few dozen varieties of locations for users to inhabit in the game world. This includes areas created at the block, building and room level. There are only a few objects in the world that one can interact with, mainly items to restore health and items to inflict damage on zombies. Other items and locations will allow the world to seem more realistic and more full of (former) life.

The narrative collection and processing conducted by the system was also incomplete, so players were not presented with narrative account of their experiences, and they could not manipulate them to better represent their version of events. The remainder of the narrative generation work will implement elements I have written for other digital literature projects and include templating, keyword-replacement, and open user-contribution. User-contributed and user-edited content create additional experiences from *Hour: 720*, so it is not surprising users felt less of a connection to the game world. Users could not input their characters' backgrounds, for example, which removes some of the depth from the player-character bond.

But this anticipated failure should not be taken as an overall failure of the first private beta tests. The development plans called for building a working interactive online world with the beginnings of narrative involvement, and as such the tests could be considered successful. If the game were completely involving without the narrative content, then a good argument could be made to leave out such material. However, the fact that the testers generally desired more narrative implies that we are on the right development path to build a more compelling game.

These additional narrative elements are significant pieces of a very large project. The game world is in place, and the system has successfully supported about a dozen players throughout a month of gameplay with no stutter. Some bugs were found and corrected, and certainly enhancements will be discovered and implemented in the final phase of the project, but *Hour: 720* already offers a working model of a virtual world. More interaction with other items and locations in the world will build more compelling

gameplay, and allow for more complicated quest-based elements. As features are added and refined, changes may be made to various gameplay mechanics, such as the chat mechanism and the mapping mechanism, which are two areas in need of attention.

The remainder of *Hour: 720* is in the process of completion. It is scheduled to relaunch for further beta testing (this time including narrative generation and manipulation) in late August, and this test will be a public beta test. I hope to gather a much larger number of participants to test the game by advertising the tests online and offline through posters and postcards. Further enhancements will be made based on user-feedback and the design plans, and the final version of the game is scheduled to launch in October 2006.

Chapter Four:
Documenting the Play

Multi-modal interactive story generation.

In addition to the traditional goals of any game to provide a compelling game experience, *Hour: 720* is conceived as a game engine that, in part, generates a complex, textured digital narrative. The game engine itself creates the structure and foundation of the digital narrative, but it does not, as in so many games, impose the developers' will on all aspects of the resulting story. Although traditional game structures such as branching plots and a sprinkling of random encounters form a part of *Hour: 720*'s narrative generation, the player is also given a significant role in the creation and editing of game-based narratives.

These roles fit into the larger game world, defined by the narrative underpinnings of the game. As the player interacts with the world a setting is built, and the player

receives information about the activities of the world beyond the scope of the player perspective. Through the use of radio broadcasts and artifacts left in the world (notes, books, diaries, journals and fliers that players can find in various locations), an overarching narrative is created that forms the basis for the user's editorial and creative input.

The intersection of games and literature

While many games have incorporated narratives, much of the critical discussion of games has focused on whether a cinematic, narrative, or game-oriented approach is best. The prevailing winds of critical study embrace a "crossdisciplinary" approach, which is evidenced by the mission statement of the peer-reviewed academic journal *Game Studies*: No individual discipline should be privileged above others. Likewise, in the creation of digital literature and interactive fictions of all types, games come in and out of the discussion freely. Janez Strehovec, in his essay "The Moving Word: Toward a Theory of Web Literary Objects," outlines a view of digital literature that includes a huge variety of forms, from hypertext to avant-garde digital concrete or sound poetry. Digital literary objects, as Strehovec labels them, move freely between all of these forms and genres because the underlying system is innately related:

> The digital word as a new media object is also a soft signifier that can be modified by software due to its very nature based on the numerical code. Such a word-object can be used as a material for new procedures of creating textscapes as well as for various manipulations within computer games. (2)

The attitude expressed by Strehovec does not begin with his work. Since Espen J. Aarseth's 1997 publication, *Cybertext: Perspectives in Ergodic Literature*, the worlds of digital literature and games have been connected in critical theory. Aarseth's conception of *ergodic* texts has become a major element of the discussion of digital literature, electronic poetry, and games in general. Aarseth defines ergodic texts as "nontrivial effort is required to allow the reader to traverse the text" (1). It is a problematic definition in a variety of ways, and has sparked a range of discussion that has resulted in some interesting analysis of various concepts of interactivity. Aarseth's work has also informed a new wave of critical interest in both digital literature and game studies.

Markku Eskelinen, one of Aarseth's most ardent acolytes, argues passionately for creators to take up Aarseth's ideas about "games as textual instruments" (Eskelinen 22). Eskelinen proceeds to detail a relationship between games, genre definitions and digital texts in light of traditional and recent theory. The notion of "textual instruments" is not new to Eskelinen's work, either: John Cayley and Noah Waldrip-Fruin have been two major proponents of the concept of "instrumental" texts, which is a critical concept that encompasses both games and digital literature objects. The idea is that texts should encourage readers to "play" in both a creative and a performative manner. The generation of the text results from the interaction of the reader with the instrument. The metaphor can be extended to include texts which gauge and record, incorporating data gathering into the creation of the text. But the crucial difference is that the reader's interaction is not merely uncovering the narrative elements hidden by the writer; rather, the generation of the narrative is accomplished through the reader's interaction with the textual instrument.

The interaction is not navigational, as in a “Choose Your Own Adventure” story or a traditional hypertext. Instrumental texts encourage readers to interact for the enjoyment and reward of the interaction itself.

This is a brief discussion of these concepts, meant only to contextualize this important point: *Hour: 720* is an instrumental text. It is a game and a narrative engine on several levels. It provides a base of material to the user and it allows the user to edit and augment that material in order to produce a narrative representative of each player's unique experience. The player and the game engine combine to create the final narrative result.

Player created and player edited texts

Hour: 720 contains scripted components which combine to create the overarching game narrative, the story of events happening well beyond the control of the players. There may be moments in which the player is able to intersect with and possibly influence these larger-scale events, but those moments are rare. However, that does not mean that each individual game experience is not potentially meaningful and important, and players are able to augment, contextualize and embellish their experiences through the manipulation and creation of game-based texts. Some narrative elements inserted by the game cannot be modified or removed, in order to provide a record of the overarching world narrative as the player has come in contact with it, but others are left up to the

player's discretion.

The character creation and management aspect of the gameplay is essential in the framework of the player created or edited elements. Each game runs for 30 days. A player may enter the game at almost any time during the 30 days (at various times there may be shut-out periods for new players or characters due to the overarching world narrative). In the course of the 30 days, a player will probably play several characters since the world of *Hour: 720* is a hostile environment full of deadly hazards. Each character is generated and at any time the player can modify the character's background story, adding or customizing history based on the player's “take” on the information given by the game (age, gender, name, occupation, etc.). The player may only play one active character at a time, but once characters have gone through their complete gameplay cycle, the collected actions of the character in the game world become accessible and editable.

For example, if a player logs in and generates a character on day one of the game, she might generate a middle-aged cop named Fred Lawson. The player could edit her character's background story at any time during her gameplay as Fred Lawson, and this can become a way to flesh out a character to encourage better role-playing. Once Fred Lawson succumbs to the dangers of the game world, the player is immediately presented with the opportunity to generate a new character and return to the game in-progress. In our example, let's say that Sheila Gordon, a retired school teacher, is the created character, and has only slightly better luck surviving than Fred had. Now the player moves on to her third character, a young man named Martin Ruiz who works with a

regional search and rescue unit. He is tasked with a specific search and rescue mission to find another player's character and return her to safety. After a long search, he discovers her dead body and returns to tell his unit the bad news. The player may perform several more rescue missions, occasionally finding survivors and often finding dead bodies. If Martin dies, the player may generate another character until the island of New Hampton is fully quarantined and the game concludes after 30 days.

Throughout the game, and through the continual creation and shedding of characters, the player in the example above has created a list of editable texts in her character management screen. She may now modify, enhance, or cut various story elements to create a textured narrative in the style she likes. This game experience narrative, coupled with the player created back story, combines to form a “chapter” of the unique player created and edited text. The effect is not unlike a novel with chapters that change perspectives, in the tradition of William Faulkner's *As I Lay Dying* or any of the other many works that have incorporated multiple perspectives, which is also known as “multi-modal” storytelling; i.e., stories told using multiple voices. A diagram of the interplay of story structures is shown in Figure 7:

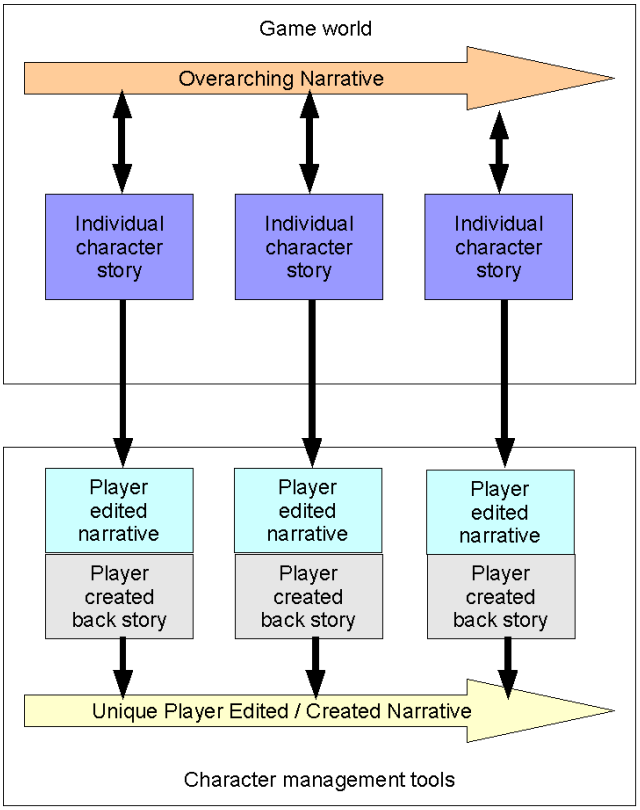

Figure 7: Interplay of narrative forces in Hour: 720.

The intent is to create a more faceted retelling of the game experience, and to encourage readers to confront the ramifications of inhabiting these different roles. The spectrum of characters a player might inhabit is rather large, and most characters will have radically different levels of success in surviving in the world due to not only their personal strengths and weaknesses, but also due to the group near which they find themselves.

This type of narrative mirrors the narratives we learn about history in educational settings, and is intended to evoke both the immediacy of the game world and lend to its reality. After all, the game world is, indeed, a virtual environment. As Ibanez et al have

written, “one of the most striking features of historical investigations is the coexistence of multiple interpretations of the same event. The same historical events can be narrated as different stories depending on the storyteller's point of view” (30). In *Hour: 720* the events of the game world become a part of the player's personal history. This is a shared history with both the other players who inhabited that instance of the game world with our example player and with the players past and present who have had experiences with a generally similar game world. Eventually, the interactions of players with the game will build a library of texts, each one replete with several interpretations of events.

The spectator experience

Once each game is over, every 30 days, players will be given a chance to modify, edit and generally preen their game narratives into a set of short stories that build to tell the story of the game world as experienced by each of the characters played throughout the past 30 days. If the player chooses, she can collect these individual character narratives into a single volume document for download and printing. This final document is referred to as the “game novel,” and is formatted automatically to look like a novelization of the game. Each document receives a custom cover and is laid out to conform with the general requirements of on-demand publishing services, should the player wish to receive an actual printed copy.

If the player chooses, the electronic document may also be included on the publicly available *Hour: 720* library. This library will contain game novels from all different instances of the game world, and both players and the general public will be

able to download print-ready copies of the game novels. In addition, the novels themselves may be remixed by the final recipient. For example, a user may edit together a collection of the most humorous, or the best-written game narratives to create a larger volume that conflates the experiences of several different players and their multitude of characters.

In this way, *Hour: 720* is a textual instrument for spectators as well as players. A base set of tools will be provided for non-players to select chapters of different game novels to combine into a single volume. The editor can input additional information to provide an explanatory introduction and secure credit for assembling the collection. This additional level of reader-participation, and the immediate connection between the assembly and output process, warps the traditional role of editor and creates a role perhaps more appropriately labeled "text-jockey." To consider these anthologies "remixes" of the original works seems appropriate, especially given the fact that the alterations do not permanently affect the source texts. Rather, the text jockeying process spins off one of many possible iterations of these texts, and that collection itself becomes a part of the source text library.

Spectators may also engage with the game world by "looking in" via an informative front page on Hour720.com, which mimics the look and feel of a local news website. The in-progress stories of different player characters will be featured, as well as media from the game world including recorded radio broadcasts and fictional press releases and statements from various local, regional and federal government officials.

The performative aspect of *Hour: 720* is not to be understated. Some players will

no doubt find great joy in garnering the spotlight as their characters (and, thus, their creative writing and gameplay) are singled out on the game's public website. Providing spectator involvement is also a way to build an audience who may not be interested in the gameplay experience, but who are compelled by a nightly news styled voyeurism and/or the subject matter and stories.

Serialized game texts

Another aspect of both the spectator and player experience with *Hour: 720* is the existence of serialized game texts. This is another method for the game to reach beyond the boundaries of the gameplay and incorporate other modes of communication. Serialized, or syndicated, media is a common feature of most websites, ranging from textual information to audio and video files (commonly known as “podcasting”). An ever-increasing number of software applications and hardware devices are capable of receiving serialized media content, and the distribution of this content is relatively low-impact in terms of required bandwidth and cost.

Hour: 720 will incorporate two types of serialized game text in the finished game. The first is a summary of game world events that is primarily targeted to active players. This general information feed allows users to get immediate updates on radio broadcasts and fictionalized media coverage of game world events. It is intended to provide a way for players to keep in close touch with the world of *Hour: 720* even when they are not able to be actively at-play. In addition to the general informative game world updates, which are available to both players and non-players, active players can configure a

customized news feed that will provide personalized updates on their active character, allowing players to become immediately informed if their character is imperiled while they are away from the game.

The second type of serialized text is targeted at both players and non-players, and creates a whole other experience of the game world. Users can receive radio broadcasts from the world of *Hour: 720* as audio downloads. These are fictional tape loops recorded daily at radio stations based in New Hampton. The broadcasts tell very different stories about the events occurring in the game world and are meant to provide both narrative context and gameplay strategy. Non-players will be able to enjoy them as short works of radio drama, while players can use the serialized audio feed as yet another way to stay connected to the game world.

It is hoped that these various narrative approaches will generate interesting textual results. When generating digital texts it becomes apparent just how difficult it is to imitate the intuitive editing and composing capability of human beings. In mainstream digital media, the composition and editing of content is done the old-fashioned way: An expert is hired to perform the task. That doesn't make the task any less time-consuming, but it does make things more difficult. New models of publishing, as seen in online websites such as *Slashdot.org* or *Digg.com*, have shown that collaborative editing

capabilities can produce a very useful assortment of quality materials. In many ways, the grandest experiment of *Hour: 720* is to see if this model of asynchronous, accretive editing can be successful in a slightly more focused environment and with a somewhat more esoteric goal: Can something approximating great literature, or even just good fiction, result from a collaborative hybrid gameplay and editing / composition process?

It could take years to find out.

Appendix

Database Outline Document ver 0.3

The following outline details the design of the *Hour: 720* game database. The fields are defined with reference to the data types used in mySQL databases. The tables are divided into two categories: The first is information that is used when each game is constructed and when each character is generated. The second set of tables are used in the actual game and are updated to correspond to user actions and manipulation.

/////////////// **information used to build each game**

table 720_jobclasses – listing of job classes for the game
- jc_id (int)
- jc_name (varchar 64) – display name
- jc_given (varchar 64) – automatic skills
- jc_primary (varchar 64) – hash of most likely skills to have i.e. 1,2,3
- jc_secondary (varchar 64) – hash of secondary skills, less likely tohave
- jc_description (text) – description, mainly for admin cohesiveness

table 720_lifestyles – listing of lifestyles for the game
- lstyle_id (int)
- lstyle_prob(int) category of how common lifestyle is (common, uncommon, rare)
- lstyle_name (varchar 64) – display name
- lstyle_desc (text) – used mainly in administration to guide story writing
- lstyle_str (varchar 12) – str effect (+1)
- lstyle_dex (varchar 12) – dex effect (i.e. -1)
- lstyle_pt (varchar 4) – pt effect (i.e. +1)
- lstyle_mt (varchar 4) – mt effect (i.e. +2)
- lstyle_skill (varchar 12) – skill_id list

table 720_skills – listing of skills in the game
- skill_id (int)
- skill_type (varchar) – auto or earned (auto doesn't show up on skills list)
- skill_name (varchar 64) – display name
- skill_desc (text) – skill description for display to all
- skill_base (int) – base to hit for skill
- skill_bars (varchar 16) – list of relevant BARs to modify skill action
- skill_barsDef (varchar 16) – list of BARs for defending against skill use
- skill_env (varchar 16) – list of environmental variables affecting skill usage

skill_reqObjFam (varchar) – item, char, npc, building, room, etc.
skill_targetObjFam (varchar) – item, char, npc, building, room, etc.
skill_effect (varchar) – list of effects action/skill has
skill_prob (int) – how common skill is

table 720_effects – listing of all effects in the game
effect_id (int)
effect_name (varchar)
effect_level (int)
effect_stat (varchar)

table 720_blocks – listing of the blocks available to games
block_id (int)
block_name (varchar 32) – for reference; generic name
block_cat (varchar 32) – shore, urban, suburban, rural
block_type (varchar 32) – urban, suburban, commercial, residential, park, etc.
block_prob(int) – how rare is it?
block_desc(text) – General description of block
block_light(int) – level of light
block_cover(int) – level of cover available
block_numBldg(int) – number of buildings on the block
block_max(int) – max number of times this block can show up

table 720_buildings – listing of buildings available to games
bldg_id (int)
bldg_name(varchar 32) – generic name
bldg_cat (varchar 32) – same as block type
bldg_type (varchar 32) – house, store, restaurant, outbuilding, etc.
bldg_prob(int) – how rare is it?
bldg_desc(text) – General description of building
bldg_light(int) – level of light
bldg_cover(int) – level of cover available
bldg_numRoom(int) – number of rooms on the building
bldg_max(int) – max number of times this building can show up
bldg_sec(int) – max security level for this building

table 720_rooms – listing of rooms available to games
room_id(int)
room_name(varchar 32) – generic name
room_cat (varchar 32) – same as building type
room_type (varchar 32) – bedroom, closet, showroom, kitchen, living room, etc.
room_prob(int) – how rare is it?
room_desc(text) – General description of room
room_light(int) – level of light
room_cover(int) – level of cover available
room_numItem(int) – number of items in the room
room_max(int) – max number of times this room can show up
room_sec(int) – base security level for this room

table 720_items – listing of items available to games
item_id (int)
item_name (varchar) – name of item.
item_cat (varchar 32) – food, manufactured, natural, fuel, etc.

item_type (varchar 32) – bread, tech, bike, branch, etc.
item_melee (int) – melee weapon effect
item_craft (int) – crafting (armor, barricade, etc.) effect
item_missle (int) – missle weapon effect
item_health (int) – effect on char health if eaten
item_description (text) – description for players to know about item and usage

Table 720_story – story fragments to be modified and customized for users
story_id (int)
story_event (varchar 32) keyword for event that prompts story to be continued
story_time (varchar 16) if fragment is dependent on time, then ts it – else=0
story_text (text) actual text of fragment

table 720_names – names used for objects in the game in the game
name_id (int)
name_object (varchar 12) – block, room, building, item
name_category (varchar 32) – category
name_type (varchar 32) – type corresponding to object
name_family – families of names (i.e. Presidents, flowers, numbers) us. streets
name_text (varchar 64) – name that user will read

table 720_radio – radio broadcasts
radio_id (int)
radio_day (int) – what day is this radio broadcast looped for
radio_text (text) – the message that is broadcast
radio_station (varchar) – station identification

table 720_pamphlets – pamphlets that are posted and dropped on the town
pamph_id (int)
pamph_time (int) – what hour are these pamphlets distributed
pamph_text (text) – text of the message on the pamphlet

////////// **information each game uses**

table 720_gameList – list of games (new game every 30 days or X number of users)
gl_id (int)
gl_begin (timestamp)
gl_end (timestamp)
gl_outcome (varchar 12) – time out? Or did the players win?

table 720_map – list of the game maps (each map is tagged with gameID)
map_id (int) – individual numeric representing each map location in sequence
map_gid (int) – id for game that this map location is a part of
map_master – location that controls this location (used to coordinate bldgs with blocks and rooms with bldgs)
map_type (varchar 8) – type of location (i.e. Block, building, room or inventory)
map_x (int) – map row of location
map_y (int) – map column of location (block of location)
map_nr (int) – current noise rating for location
map_security (int) – security of location
map_maxSec (int) – max security of location
map_name (varchar 64) – name of location
map_desc (text) – description of location

map_cover (int) – level of cover avail for hiding
map_light (int) – level of light in location
map_zoom (varchar 12) – zoom level of map location-- block, bldg, room

table 720_gameItems – list of the items used in the game
gItem_id (int)
gItem_mid (int) map_id for current location of item
gItem_cat (varchar 32) – food, manufactured, natural, fuel, etc.
gItem_type (varchar 32) – bread, tech, bike, branch, etc.
gItem_effect1 (varchar 32) – health, skill, damage, construction, repair, etc.
gItem_name (varchar 32) – name of item
gItem_desc (text) – description for players to know about item and usage
gItem_target (varchar 32) – self, other, enemy, item, building
gItem_action (varchar 32) – heal, hurt, combine
gItem_quality (int) – base quality of item, later will generate specific quality
gItem_scarcity (int) – how rare is the item?

table 720_users – list of registered users and data
u_id (int)
u_admin (int) – true/false if you have admin privs
u_name (varchar 12)
u_pass (varchar 12)
u_email (varchar 128)
u_firstName (varchar 16)
u_lastName (varchar 16)
u_registeredDate (timestamp)
u_lastLogin (timestamp)
u_gameList (text) – list of game IDs user has played in
u_ageCheck (int) – indicate that the user claims to be over 17

table 720_zombies – list of generated zombies
z_id (int)
z_name (varchar 48)
z_gender (varchar 1)
z_health (int)
z_mh(int)
z_description (desc)
z_target(txt) – the current player the zombie has targeted
z_date (date)
z_removed (int)

table 720_gameactions – list of actions taken during each game
ga_id – game action ID
ga_gid – game ID
ga_pcID – ID of player character attached to action
ga_timestamp – timestamp for action taken
ga_text – text of action report

table 720_gamePCs – list of player characters
c_id (int)

c_gid (int) – game ID
c_uid (int) – user ID
c_firstName (varchar 24)
c_lastName (varchar 24)
c_creationDate (timestamp)
c_lastMoveTS (timestamp) – time of last move char made
c_loc – character's current location
c_dob (date)
c_gender (varchar 1)
c_age (int)
c_ageGroup (varchar 4)
c_profession (varchar 32)
c_job (varchar 32)
c_str (int)
c_dex (int)
c_mt (int)
c_pt (int)
c_hp (int)
c_mh(int)
c_lstyle (varchar 32)
c_bodyType (varchar 12)
c_skills – hash of skills and level "skillID,level-skillID,level" i.e. 23,2-12,1
c_inventory (varchar 255) – hash of gItem_id in character inventory
c_infected (int) – on/off whether or not char is infected
c_zombie (int) – on/off whether or not char is a zombie
c_active (int) – on/off whether or not a char can be played
c_friends (text) – hash of friends made in the game

table 720_charStories – story fragments tied to each char (number consecutive to"end")s
cStory_id (int) – id of this story fragment
cStory_cid (int) – id for character related to this fragment
cStory_text (text) – text of story fragment
cStory_date (timestamp) – timestamp for when story fragment was written

table 720_gamechat – some table to hold chat info and/or messages
chat_id (int)
chat_loc (int) – location of chat
chat_speaker (varchar) – name of speaker
chat_time (varchar) – time line was spoken
chat_text (text) – text of spoken line

Code Samples

The following code samples are provided to illustrate some key features of the software or style of programming. These are provided as reference, and are only snippets of the actual code.

Javascript (AJAX) sample:

This sample is based on the samples available at the *Surebert* library website, surebert.com. The Javascript used on the client side is as minimal as possible, which will allow the game to be more easily ported to other display platforms. The function below creates a new AJAX object called “js” that can communicate with the Web server to dynamically render information to the player. The reference to “surebert” is handled by the *Surebert* code library, included on the first line. The function “newSession()” is executed when the page loads for the first time and prompts the eventParser module to render the information needed to initialize a new play session of the game. The following functions are each called whenever the player chooses to perform the corresponding action. Thus, “searchLocation()” is called when the player clicks on the search button, and “useItem()” is called when the player clicks to use an item.

```
<script type="text/javascript" src="js/Surebert.js"></script>
<script language="JavaScript" type="text/JavaScript">
js  = new surebert;
js.format = "js";
function newSession(){
```

```
        js.fetch('eventParser.php?a=new');
        setTimeout("timedResponse();", 50000);
}
function searchLocation(){
        myURL = "eventParser.php?a=search";
        js.fetch(myURL);
}
function takeItem(itemKey,itemName){
        myURL = "eventParser.php?a=take&itemID="+itemKey+"&itemName="+itemName;
        js.fetch(myURL);
}
function dropItem(itemKey){
        myURL = "eventParser.php?a=drop&itemID="+itemKey;
        js.fetch(myURL);
}
function useItem(itemKey, useAction){
        myURL = "eventParser.php?a=use&itemID="+itemKey+"&useAction="+useAction;
        js.fetch(myURL);
}
</script>
```

PHP code sample:

The following code is a small example of the way the eventParser.php class handles a call from the player's computer. First, the eventParser class is defined. In this snippet, only the “newGameSession()” function is shown here to provide some continuity. Once the eventParser class has been defined, the last few lines simply invoke the class to take action depending on a control variable (referred to as $_GET['a'] in PHP code) sent to the eventParser.php file from the player's computer.

```
class eventParser {
... code removed ...
        function newGameSession(){
        //establish and return data for a new game session
                //initialize environmental vars needed in figuring things out
                $this->loadChar();
                $_SESSION['lastCheck']=date("U");
                $_SESSION['loc']=$_SESSION['char']['c_loc'];
                $this->moveChar();
                $this->checkOthers();
                $this->figureSleep();
                $this->getTimeDate();
                $_SESSION['char']['c_lastMoveTS']=date("U");
        }
... code removed ...
}
session_start();
$report = new eventParser;
$report->makeReport();
```

Sample output from eventParser.php

Below is a sample of the data that is passed from the server to the Web browser in the background, unseen by the user until the screen is updated. The information is encoded to be interpreted by the Javascript running on the page. Thus, this is an example of live Javascript being executed directly from the server, which is the main method used by *Hour: 720* to provide dynamic game feedback to the player. Bold, italiced text indicates explanations added by the author for readability and not present in the actual machine transmissions.

Sample of returned Javascript data:

The code below is actual code returned from the server in a sample AJAX exchange. The bolded lines provide explanation of what is happening in the actual program code and can be read as a "pseudo-code" version of this exchange.

```
hex=51;
bhex=204;
document.getElementById("eventText").style.color=
"rgb("+hex+","+hex+","+hex+")";document.getElementById("eventText").style.backgroundColor
= "rgb("+bhex+","+bhex+","+bhex+")";
```

Set the background color shift of the main feedback area.

```
document.getElementById('eventText').innerHTML = unescape("%3Chr%20%2F%3E");
```

Send divider bar to separate new feedback from old feedback

```
document.getElementById('healthIcon').innerHTML =
unescape("%3Cimg%20src%3D%27gameImages%2FhealthIcons%2Fmh_1.png%27%20title%3D
%27MH%3A%2031%27%20%2F%3E%3Cbr%20%2F%3E%3Cimg%20src%3D%27gameImage
s%2FhealthIcons%2Fwom1_h1.png%27%20%20title%3D%27HP%3A%2030%27%20%2F%3E"
);
```

Display the proper health icon for the character type and current health levels.

document.getElementById('age_name').innerHTML =
unescape("%3Ch3%3EEmerita%20Allenbach%3Cbr%20%2F%3EAge%3A%2027%3C%2Fh3%3E");document.getElementById('mapTable').innerHTML =
unescape("%3Cp%3E%3Ctable%20width%3D%22300%22%20border%3D%220

... code removed ...

%20href%3D%27%23%27%3ESouth%3C%2Fa%3E%3C%2Ftd%3E%3Ctd%3E%3C%2Ftd%3E%3C%2Ftr%3E%3C%2Ftable%3E%3C%2Fp%3E");

The large chunk of text above represents the code for the navigation segment of the user interface. The code is interpreted into a complex table arrangement and varies depending on where the character is located.

document.getElementById('eventText').innerHTML =
unescape("%3Cp%3EYou%20have%20entered%20Rowan%20Hills%20Shopping%20Plaza.%20A%20suburban%20shopping%20mall%20and%20assorted%20ancillary%20shops%20are%20located%20here%20with%20a%20big%20parking%20lot%20and%20lots%20of%20stores.%3C%2Fp%3E") + document.getElementById('eventText').innerHTML;

The code above is a basic description of the present location.

document.getElementById('map_name').innerHTML =
unescape("Rowan%20Hills%20Shopping%20Plaza
Day: 27 Hour: 622");

The name of the area and the time and date in the game are sent.

document.getElementById('othersZombies').innerHTML =
unescape("%3Cp%3EThere%20are%20no%20others%20here.%3C%2Fp%3E%3Cp%3EThere%20are%20no%20zombies%20present.%3C%2Fp%3E");

A report of other player characters and zombies present in this location is sent.

document.getElementById('attackOptions').innerHTML =
unescape("%3Cinput%20name%3D%22attackButton%22%20type%3D%22button%22%20id%3D%22attackButton%22%20value%3D%22Attack%22%20onclick%3D%27attackProcess%28%29%3B%27%2F%3E%20%0D%0A%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%3Cselect%20name%3D%22target%22%3E%3Coption%20value%3D%27pc%2C%27%20selected%3D%27selected%27%3EEmerita%20Allenbach%20%28pc%29%3C%2Foption%3E%3C%2Fselect%3E%20%0D%0A%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20with%3Cselect%20name%3D%22weapon%22%3E%3Coption%20value%3D%27melee_0_hands%27%3Ebare%20hands%20%28melee%29%3C%2Foption%3E%3C%2Fselect%3E");

The actions form is processed and returned according to what actions are possible and the present situation.

fadetext();

The command is sent to flash the text that has changed to draw users' attention to the updated information.

Works Cited and Consulted

Aarseth, Espen J. "A multidimensional typology of games." (2003): 7 pgs. *Digital Games Research Association*. 17 Mar 2006. <http://www.digra.org/dl/display_html?chid=http://www.digra.org/dl/db/05163.52481>.

Aarseth, Espen J. *Cybertext: Perspectives on Ergodic Literature*. Baltimore: Johns Hopkins University Press, 1997.

Aarseth, Espen J. "Genre Trouble: Narrativism and the Art of Simulation." 21 May (2004). *Electronic Book Review*. 17 Mar 2006. <http://www.electronicbookreview.com/thread/firstperson/vigilant>.

Allen, Dean et al.. Textpattern. Dean Allen, 2006.

America On-Line. Neverwinter Nights. AOL, 1991.

Bartle, Richard and Roy Trubshaw. Multi-User Dungeon (MUD). Bartle and Trubshaw, 1978.

Castronova, Edward. "On Virtual Economies." 3.2 (2003): 20 pgs. *Game Studies*. 17 Mar 2006. <http://www.gamestudies.org/0302/castronova/>.

Curtis, Pavel. LambdaMoo. Xerox-PARC, 1990.

Daglow, Don. Dungeon. Don Daglow, 1975.

Dibbell, Julian. *My Tiny Life: Crime and Passion in a Virtual World*. New York: Henry Holt & Company, 1998.

Eskelinen, Markku. "Six Problems in Search of a Solution." 2004.03 (2004). *Dichtung-Digital*. 17 Mar 2006. <http://www.dichtung-digital.com/2004/3/Eskelinen/>.

Filiciak, Miroslav. "Hyperidentities: Postmodern Identity Patterns in Massively Multiplayer Online Role-Playing Games." *The Videogame Theory Reader*. Noah Waldrip-Fruin and Bernard Perron. Routledge, 2003. 87-102.

Garriott, Richard and Ralph Koster. Ulitma Online. Electronic Arts, 1997.

Glazier, Loss Pequeno. *Digital Poetics : Hypertext, Visual-Kinetic Text and Writing in Programmable Media*. Alabama: University Alabama Press, 2001.

Gygax, Gary and Dave Arneson. *Dungeons and Dragons*. USA: TSR Inc, 1974.

Hayles, N. Katherine. *Writing Machines*. Cambridge: MIT Press, 2002.

Ibanez, J and R. Aylett and R. Ruiz-Rodarte. "Storytelling in virtual environments." *Virtual Reality*. . , 2003. 30-42.

Kline, Stephen and Nick Dyer-Witheford and Greig de Peuter. *Digital Play: The interaction of technology, culture and marketing*. Quebec: McGill-Queen's University Press, 2003.

Lagney, Patric. Sociolotron. Sociolotronics, 2006.

Mateas, Michael. "A Preliminary Poetics for Interactive Drama and Games." *First Person*. Noah Waldrip-Fruin and Pat Harrigan. MIT Press, 2004. 19-33.

Mortensen, Torill. "Playing With Players: Potential Methodologies for MUDs." 02.01 (2002): 12 pgs. *Game Studies*. 17 Mar 2006. <http://www.gamestudies.org/0102/mortensen/>.

Poole, Steven. *Trigger Happy*. New York: Arcade Publishing, 2000.

Ryan, Marie-Laure. "Beyond Myth and Metaphor: The Case of Narrative in Digital Media." 01.01 (2001): 16 pgs. *Game Studies*. 17 Mar 2006. <http://www.gamestudies.org/0101/ryan/>.

Salen, Katie and Eric Zimmerman. *Rules of Play: Game Design Fundamentals*. Cambridge: MIT Press, 2004.

Steinkuehler, Constance A. "Where everybody knows your (screen) name: Online games as "third places."." (2006): 31 pgs. *CA Steinkuehler MMOG Research Site*. 17 Mar 2006. <http://website.education.wisc.edu/steinkuehler>.

Steinkuhler, Constance A. "Why game (culture) studies now?." 01.01 (2006): 6 pgs. *Games and Culture*. 17 Mar 2006. <http://website.education.wisc.edu/steinkuehler/>.

Strehovec, Janez. "The Moving Word: Toward a Theory of Web Literary Objects." (2001): 16 pgs. *Cybertext Yearbook 2000*. 22 Nov 2003. <http://www2.arnes.si/~ljzpubs1/eseji.htm>.

Strehovec, Janez. "Theories of Cyberculture and Internet Textuality." 05 Apr 2002. *Internet Culture and Internet Textuality*. 17 Mar 2006. <http://www2.arnes.si/~ljzpubs1/theories.htm>.

Turkle, Sherry. *Life on the Screen*. New York: Simon and Schuster, 1995.

Valve Software. "Valve - Awards We Have Won." 2006. *Valve Software*. 08 Apr 2006. <http://www.valvesoftware.com/awards.html>.

Visco, Paul. Surebert. P:Labs, 2005.

Waldrip-Fruin, Noah and Pat Harrigan, eds. *First Person: New Media as Story, Performance, and Game*. Cambridge: MIT Press, 2004.

Wallace, Mark. "In Celebration of the Inner Rogue." Issue 30 (2005): 3 pgs. *The Escapist*. 17 Mar 2006. <http://www.escapistmagazine.com/issue/30/3>.

Wolf, Mark JP. "Genre and the Video Game." *The Medium of the Video Game*. Mark

JP Wolf. University of Texas Press, 2001. 113-34.

Wolf, Mark JP and Bernard Perron. *The Video Game Theory Reader*. New York: Routledge, 2003.

Woodcock, Bruce Sterling. "MMOG Chart." May 21 (2006). . . <http://www.mmogchart.com>.